Micro-Routing with Graph Neural Cellular Automata for Sub-Millisecond Agent Tool Selection

Julian M. Kleber
Independent Researcher
julian.m.kleber@gmail.com

November 2026

Abstract

Agent harnesses choose the next tool on every turn, usually with a multi-billion-parameter language model. As a small learned router, a Graph Neural Cellular Automaton (GNCA) holds the goal and the last four tool steps in five cells. It runs K weight-tied gated message-passing steps over a call-tree graph and reads the next tool from the final cell states (Algorithm 1). We derive the update rule from random-walk diffusion and state its exact structural properties. Lean 4 proofs (28 theorems, no `sorry`) cover receptive-field separation and decision-tree simulation; they do not certify the trained network. On four benchmarks (Hermes agent traces, BFCL, ToolBench, API-Bank) we tune 13 models with the same budget, select each on multi-seed validation and test with five seeds. The tree GNCA is competitive, not more accurate. Pooled over benchmarks it shows no significant difference from HistGradientBoosting (+0.10 pp, 95% CI $[-1.05, +1.25]$), an MLP, a small Transformer or an untied GNN, and it is ahead of k-NN retrieval (+2.21 pp). Gradient boosting on the same cell features is ahead of it on Hermes (-1.10 pp after Holm correction). Removing weight tying, the gate, the Laplacian block or the row normalisation has no detectable effect. On held-out private sessions every model loses 11.76–14.40 pp. On one CPU thread the GNCA's inference takes 0.559 ms per turn; our unoptimised Python featuriser adds 7.65 ms of preprocessing, which every feature-based baseline also needs. Predictive entropy detects routing errors better than our step-norm escalation score on 3 of 4 benchmarks. All numbers are generated from public result files.

1. Introduction

An autonomous agent works in a loop: it reads feedback from its environment (compiler output, file diffs, test logs) and picks the next action. Most of these picks are routine. Nervous systems handle routine motor actions with fast reflex arcs instead of deliberation; agent harnesses, in contrast, send every turn through a large autoregressive model. That costs latency and memory traffic on every turn and fills the context window with tool schemas. On our hardware two local models need 378 ms (Qwen3.5-9B) and 848 ms (Gemma-4-12B) per zero-shot routing decision on the Hermes test split. They reach only 35.01% and 38.02% top-1, below the majority-class rate of 52.86%.

A small supervised router trained on the harness's own traces is the obvious alternative, with the large model kept as a fallback for uncertain turns. Neural cellular automata (NCAs) [1] apply one learned local rule repeatedly; graph variants [2,3] run it on arbitrary graphs. An agent trace has natural cells (the goal and recent steps) and a small graph that can encode how the goal reaches the history, so a GNCA is a plausible router: tiny, weight-tied, and analysable. Earlier NCAs for text [6] work on 1D chains, where the goal needs $V-1$ steps to reach the latest cell.

Contributions.

- A GNCA next-tool router over five trace cells with chain, star, tree and learned topologies, with the update rule derived from random-walk diffusion, its spectrum and its exact parameter count (Section 3).
- Lean 4 formalisations of the structural facts behind the design, with an axiom audit and an explicit statement of what they do not cover; an empirical Jacobian check of the contraction hypothesis (Section 4).
- A four-benchmark comparison of 13 models, including a small Transformer, an untied GNN that isolates weight tying, and k-NN retrieval. Every model is tuned with the same random-search budget, selected on the three-seed mean validation score and tested with five seeds. Paired tests resample test turns with seeds kept together, use Holm correction and add a random-effects pooled estimate (Sections 5–6.1).
- Component ablations, an out-of-distribution test on held-out private sessions, a session-level proxy for task success, CPU inference latency with the featurisation cost reported separately, and an escalation analysis of the primary model (Sections 6.2–6.3).

The headline is modest, and we report it as such. With equal tuning and five seeds the GNCA shows no significant pooled difference from gradient boosting, an MLP, a Transformer or an untied GNN (Table 3). It is ahead of k-NN retrieval, and gradient boosting on the same cell features is ahead of it on Hermes. Its own components (weight tying, gate, Laplacian block, normalisation) have no detectable effect on accuracy, and its step-norm esca-

lation term does not help. Our claim is limited to this: on these benchmarks the pooled differences between a new architecture and methods refined for over a decade are not significant (Table 3), which we consider a useful early-stage result. Version 7.6 of this manuscript claimed more than the result files support; Appendix E lists what was removed or replaced.

Scope. All accuracy claims concern turn-level next-tool accuracy on held-out splits; Section 6.2 adds an out-of-distribution split, a session-level proxy and latency. The primary model (tree GNCA, K=4, label smoothing) and the twelve primary comparisons are those coded in the analysis script. The main comparison (five seeds, multi-seed selection) reuses them and adds a third family for the new baselines. There is no pre-registration, and earlier exploratory runs on Hermes informed these choices, so the tests are not confirmatory in the strict sense. Code, splits, result files, the scripts that generate every number, table and figure, and the Lean proofs are public (MIT licence).

2. Related Work

Neural cellular automata. Growing NCAs [1] learn local update rules on image grids that self-organise and repair. Learning Graph Cellular Automata [2] move the rule to arbitrary graphs and fit it to reproduce given dynamics (e.g. Voronoi or flocking rules); E(n)-equivariant GNCAs [3] add geometric equivariance for point clouds. NCAs have also been used for reasoning tasks [4], brain-inspired wiring [5] and language modelling [6,7]. Our cell rule is a standard GNCA update with the NCA perception stack (identity, neighbourhood average, Laplacian).

Weight-tied message passing. Sharing one propagation function across steps is older than graph NCAs: gated graph neural networks (GGNN) [8] unroll a shared GRU update for a fixed number of steps, the message-passing framework [9] covers tied and untied variants, and Universal Transformers [10] and deep equilibrium models [11] tie weights across depth in sequence models. The GNCA used here is a member of this family. It differs from GGNN in three concrete ways: the observation o_i is re-read at every step instead of only initialising the state, the update is a gated residual that starts at the identity (W_2=0), and the perception is the NCA stack on a row-normalised operator. We test the weight tying directly with an untied GNN that is otherwise identical (Section 6).

What is new. Table 1 separates what we take from prior work from what this paper adds. To our knowledge no earlier work uses a graph NCA as an in-loop next-tool router. The algorithm is the combination in Algorithm 1: the agent trace as a five-cell graph (goal plus the last four steps, each with its result), the call-tree operator over those cells, K weight-tied gated steps with the observation re-injected, a flat readout and an entropy hand-off. The building blocks are known; the contribution is this router, its exact structural properties (Proposition 3, Theorem 2) and a controlled comparison with mature baselines.

Neural and soft decision trees. Deep neural decision forests [15], distilled soft trees [16] and deep neural decision trees (DNDT) [17] make tree routing differentiable. DNDT builds leaves from Kronecker products of soft bins, so its size grows exponentially with the number of features. Our Lean result about trees (Theorem 2) is a simulation statement in the same spirit: one shared cellular step on a tree evaluates any decision tree in depth-many steps.

Routing in LLM systems. FrugalGPT [18] and RouteLLM [19] route a whole query between models of different cost. AgentWeave [20] routes before reasoning in function calling, and recent work evaluates System-1 decision models for agent harnesses with paired, self-audited tests [21]. Small function-calling models for edge devices (Octopus v2 [22], TinyAgent [23], Hammer [24]) generate the call itself, and tool retrievers rank tools by relevance to the query [25]. We address a narrower problem: predicting the next tool from the trace inside the loop, with an optional hand-off to a larger model when the router is unsure.

Verification of neural networks. SMT- and bound-propagation verifiers such as Marabou [32] and Beta-CROWN [33] certify input-output properties of trained feed-forward networks. Our Lean proofs are of a different kind: they establish properties of idealised graph automata, not of trained weights (Section 4).

Benchmarks. We use Berkeley Function Calling Leaderboard data [27] from the Gorilla project [26], ToolBench trajectories [28], API-Bank dialogues [29] and public Hermes agent traces [30].

3. Task Formulation & Architecture

Operational context and action. At turn t the agent sees a context $\mathbf{s}_t = (\mathbf{g}, \mathcal{H}_t)$. It consists of the user goal $\mathbf{g}$ and the trace $\mathcal{H}_t = \{(u_{t-k}, r_{t-k})\}_{k=1}^{4}$ of the last four tool calls u with their results r. The router predicts the next tool $y \in \mathcal{C}$ from a shared vocabulary of $|\mathcal{C}|$=13 tools (file read/edit, shell, search, respond, ...); a benchmark uses the subset that occurs in it. It also returns an escalation score s_{esc} (Section 3.4) that decides whether the turn is handed to a large model. Argument filling is left to the downstream model.

3.1. Trace graph and cell features

The graph has V=5 cells. Cell 0 holds the goal; cells $1, \ldots, 4$ hold the last four steps, right-aligned so that cell 4 is the most recent step $t-1$ (cells without a step

Table 1: Positioning: what the tree-GNCA router takes from prior work and what it adds. "Tied": one set of update weights for all steps.

	Growing NCA [1]	Graph NCAs [2,3]	GGNN / MPNN [8,9]	Tree ensembles, neural trees [15,17]	This work (Algorithm 1)
Cells / nodes	Pixels on a grid	Graph nodes, points in space	Graph nodes	Input features	Goal + last four tool steps of an agent trace
Graph	Fixed grid	Given graph	Given graph	–	Fixed call tree (or chain, star, learned) over trace cells
Update	Tied local rule, stochastic cell updates	Tied rule	GGNN: tied GRU; MPNN: tied or untied	Axis-aligned splits	Tied gated residual from identity; observation re-read every step
Objective	Grow / regenerate a target image	Reproduce given dynamics; pattern formation	Graph or node labels	Labels	Next-tool classification with entropy hand-off
Parameters vs. steps K	Constant	Constant	GGNN constant	Grows with depth and trees	Constant except KH step embeddings (Prop. 3)
Formal results	–	Equivariance (E(n))	–	–	Lean: receptive fields, tree simulation; exact locality and automorphism equivariance (Prop. 3)
Taken from prior work	Perception stack (identity, average, Laplacian); weight tying; gated residual updates; random-walk operator [31]; TF-IDF/SVD features				

stay zero). Each cell carries D=146 observation channels:

- channels 1–128: a TF-IDF/truncated-SVD embedding fitted on the training goals of each benchmark and seed. Cell 0 embeds the goal text; step cells embed the text "tool + result" in the same basis;
- channels 129–132: four scalars. Goal cell: number of steps, log length of the last result, last-result error flag, log goal length. Step cells: position, log result length, error flag (result contains "error" or "fail"), active flag;
- channels 133–145: one-hot of the tool called in that step (zero for the goal cell);
- channel 146: occupancy mask.

Each cell also carries a hidden state $h_i \in \mathbb{R}^H$, H=16, initialised to zero, so the full cell vector has $D_{\text{cell}} = D + H = 162$ channels.

3.2. Topologies and the random-walk operator

Let $S \in \{0,1\}^{V\times V}$ be a symmetric 0/1 pattern with self-loops ($S_{ii} = 1$) and $Q = \text{diag}(S\mathbf{1})$ its degree matrix. The code uses the row-normalised operator

$$A = Q^{-1}S, \qquad A_{ij} \geq 0, \quad \textstyle\sum_j A_{ij} = 1. \tag{1}$$

Chain: $S_{ij} = 1$ for $|i-j| \leq 1$. *Star*: cell 0 linked to every cell. *Tree*: cell $i > 0$ linked to its parent $\lfloor (i-1)/2 \rfloor$, which gives

$$A_{\text{tree}} = \begin{pmatrix} 1/3 & 1/3 & 1/3 & 0 & 0 \\ 1/4 & 1/4 & 0 & 1/4 & 1/4 \\ 1/2 & 0 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 1/2 & 0 & 0 & 1/2 \end{pmatrix}, \tag{2}$$

so the two most recent steps are siblings under $t-4$. *Learned*: $A = \text{softmax}_{\text{row}}(\alpha/\kappa)$ with free logits $\alpha \in \mathbb{R}^{5\times5}$ initialised at $\log(A_{\text{tree}} + 10^{-4})$ and temperature κ=0.5.

Sign convention and spectrum. We write $L_{\text{rw}} = I - A$ for the random-walk Laplacian [31]. We use the drift operator $-L_{\text{rw}} = A - I$; its i-th row is $\sum_j A_{ij}(x_j - x_i)$, the deviation of the neighbourhood mean from cell i.

Proposition 1 (Spectrum of the fixed operators)**.** *Let A be as in Eq. (1). (i) The eigenvalues of A are real and lie in $[-1,1]$, with $A\mathbf{1} = \mathbf{1}$. (ii) The eigenvalues of $A - I$ are real and lie in $[-2,0]$. (iii) On a connected pattern, 0 is a simple eigenvalue of $A-I$ with eigenvector $\mathbf{1}$ (consensus).*

Proof. $A = Q^{-1}S$ is similar to $Q^{-1/2}SQ^{-1/2}$, which is symmetric, so the spectrum of A is real. Row-stochasticity gives $\|A\|_\infty = 1$, hence $|\lambda| \leq 1$, and $A\mathbf{1} = \mathbf{1}$. Shifting by $-I$ gives (ii). For (iii), A is the transition matrix of a random walk on a connected graph with self-loops, which is irreducible and aperiodic; by Perron–Frobenius the eigenvalue 1 is simple. □

For the matrices built by the code the second-largest eigenvalue of A is 0.833 (chain), 0.500 (star) and 0.766 (tree), and the smallest is −0.260, −0.300 and −0.280. A pure linear diffusion $x \leftarrow Ax$ therefore forgets non-consensus components fastest on the star; the learned update below is not restricted to diffusion.

Is an undirected graph a source of leakage? A common concern is that a graph operator over a sequence must be lower-triangular (causal) to avoid future-to-past contamination. For linear filters the general statement is true.

Proposition 2 (Leakage under linear filtering)**.** *Let $X_1, \ldots, X_V$ be scalar events in temporal order with innovations $\eta_k = X_k - \mathbb{E}[X_k \mid X_{<k}]$ that are independent of $X_{<k}$ and have $\text{Var}(\eta_k) = \sigma_k^2 > 0$, and let*

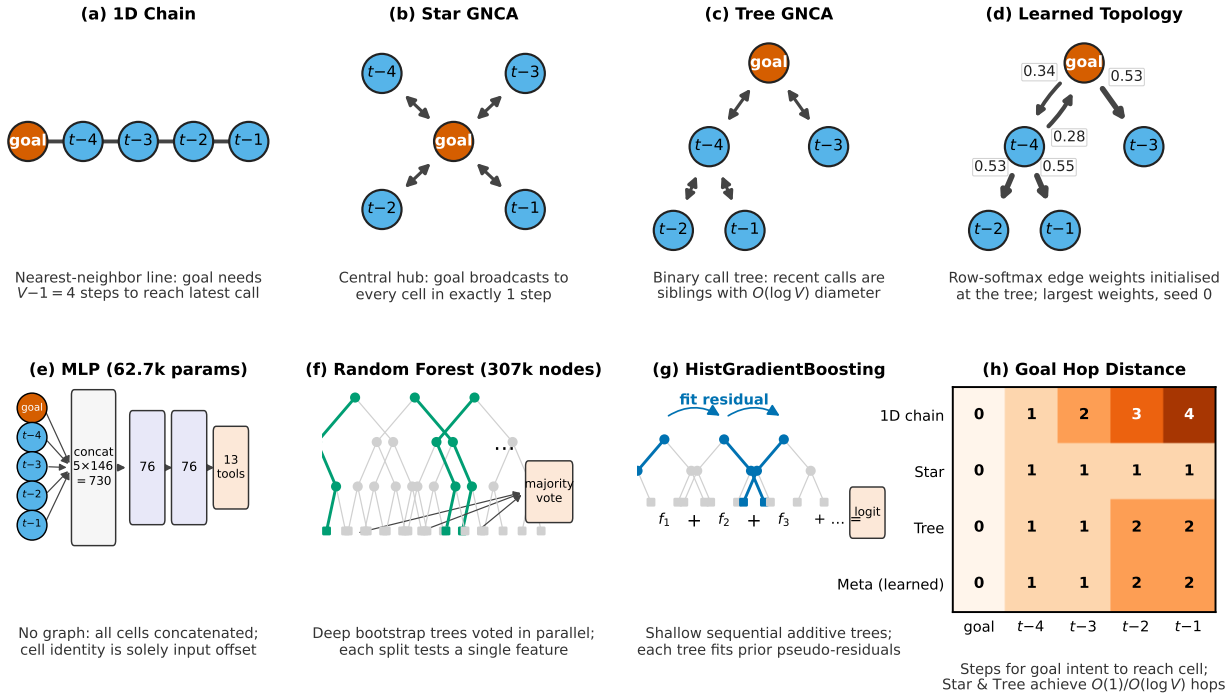


Figure 1: Information-flow graphs over five cells (goal cell and the last four steps $t-4, \ldots, t-1$), drawn from the adjacency matrices built by the training code, with the baselines for reference. (a)–(c) Fixed topologies; edges are undirected. (d) Learned topology: largest row-softmax weights of seed 0. (e)–(g) MLP, Random Forest and HistGradientBoosting baselines (parameter and node counts from the result files). (h) Number of steps the goal needs to reach each cell.

$\tilde{X} = MX$. *Then* $\tilde{X}_i$ *is a function of* $X_{\le i}$ *for every* i *if and only if* $M_{ik} = 0$ *for all* $k > i$. *If not, let* k *be the largest index with* $k > i$ *and* $M_{ik} \neq 0$; *then* $\mathrm{Cov}(\tilde{X}_i, \eta_k \mid X_{\le i}) = M_{ik}\sigma_k^2 \neq 0$.

Proof. $\tilde{X}_i = \sum_{j \le i} M_{ij} X_j + \sum_{j>i} M_{ij} X_j$. If the second sum vanishes, $\tilde{X}_i$ is measurable with respect to $X_{\le i}$. Otherwise write $X_k = \mathbb{E}[X_k \mid X_{<k}] + \eta_k$. Because k is the largest index with $M_{ik} \neq 0$, every other term of $\tilde{X}_i$ is a function of $X_{<k}$ and hence independent of η_k. Therefore $\mathrm{Cov}(\tilde{X}_i, \eta_k \mid X_{\le i}) = M_{ik}\,\mathrm{Var}(\eta_k) \neq 0$ and $\tilde{X}_i$ is not a function of $X_{\le i}$. A two-node example: with $S = \frac{1}{2}\mathbf{1}\mathbf{1}^\top$, $\tilde{X}_1 = \frac{1}{2}X_1 + \frac{1}{2}X_2$ carries $\frac{1}{2}\eta_2$. □

The proposition does not apply to our router. All five cells hold the goal and *past* steps that are observed when the prediction is made, and the label (the next tool) is in no cell. An undirected edge between $t-4$ and $t-1$ mixes two observed events; it carries no information from after the decision point. Directed masking would matter only if cells held events later than the decision, for example when training on full trajectories with a label at every step. We therefore use undirected graphs, as the code does, and make no causal-filtering claim.

3.3. Cellular update rule

From diffusion to the perception stack. The linear diffusion $\dot{\mathbf{x}} = (A - I)\mathbf{x}$ discretised with unit step gives $\mathbf{x}^{(n+1)} = A\mathbf{x}^{(n)}$, i.e. repeated averaging. NCAs [1] instead pass the identity, a smoothing and a difference filter to a learned per-cell network. On the graph these are x, Ax and $(A-I)x = Ax - x$:

$$p_i^{(n)} = \left[\, x_i^{(n)},\ [A\mathbf{x}^{(n)}]_i,\ [(A-I)\mathbf{x}^{(n)}]_i \,\right] \in \mathbb{R}^{3D_{\text{cell}}}, \quad (3)$$

with $x_i^{(n)} = [o_i;\, h_i^{(n)} + e_n]$, o_i the fixed observation of cell i and $e_n \in \mathbb{R}^H$ a learned embedding of step n. The third block is the difference of the other two, so it adds no information; we keep it because it is the standard NCA perception stack [1] and costs only weights. Removing it has no detectable effect on accuracy (Section 6.3).

Gated residual update. A shared two-layer network (a 1×1 convolution over cells) maps the perception to a candidate update and a gate,

$$\begin{aligned} [\delta_i;\, g_i] &= W_2\, \mathrm{ReLU}(W_1 p_i^{(n)} + b_1), \\ h_i^{(n+1)} &= h_i^{(n)} + \sigma(g_i) \odot \delta_i, \end{aligned} \quad (4)$$

with $W_1 \in \mathbb{R}^{96 \times 3D_{\text{cell}}}$ and $W_2 \in \mathbb{R}^{2H \times 96}$. W_2 is initialised at zero, so training starts from the identity map; σ is the logistic function. The rule is shared across cells and steps. After K steps the tool head reads $\hat{y} = \mathrm{softmax}(W_{\text{tool}}\,\mathrm{vec}(\mathbf{h}^{(K)}) + b_{\text{tool}})$ from the flattened 5×16 hidden states. There is no stochastic cell masking,

damping factor or halting in the primary model; inference is deterministic.

Worked information flow on the tree (K=4). At step 1 the leaf cells $t-2$ and $t-1$ (results, error flags) update from their parent $t-4$ and themselves, and $t-4$ averages over the goal and both leaves. At step 2 the goal cell, which averages over $t-4$ and $t-3$, receives the leaves' information through $t-4$. Steps 3–4 send goal-conditioned state back to the leaves. The goal reaches every cell within two steps and every pair of cells is connected within three (Figure 1h); on the chain the goal needs four steps to reach $t-1$.

Parameter count. With D_{mid}=96 the module has

$$\begin{aligned} N &= \underbrace{96 \cdot 3D_{\text{cell}} + 96}_{W_1, b_1} + \underbrace{2H \cdot 96}_{W_2} + \underbrace{KH}_{e_n} \\ &\quad + \underbrace{VH|\mathcal{C}| + |\mathcal{C}|}_{\text{tool head}} + N_{\text{aux}} \qquad (5) \\ &= 46{,}752 + 3{,}072 + 64 + 1{,}053 + N_{\text{aux}}, \end{aligned}$$

where $3D_{\text{cell}} = 486$. The PyTorch module also contains a bilinear argument head ($32 \cdot H^2 + 32 = 8{,}224$) and an abstain gate ($H + 1 = 17$), together N_{aux}. The code counts them in its totals (N = 59,182 for K=4, 59,246 for K=8, 59,207 with learned logits). They receive no gradient, because the loss is cross-entropy on the tool head only. The trained part has 50,941 parameters. The weights do not depend on K except through e_n, and the history length is fixed by V.

3.4. Escalation score

For hand-off we score each turn with

$$s_{\text{esc}} = \underbrace{\frac{\mathcal{H}(\hat{y})}{\ln|\mathcal{C}|}}_{\bar{\mathcal{H}} \in [0,1]} + \beta \underbrace{\frac{1}{V}\sum_{i=1}^{V} \frac{\|h_i^{(K)} - h_i^{(K-1)}\|_2}{\sqrt{H}}}_{\bar{s}}, \qquad (6)$$

and escalate when $s_{\text{esc}} > \theta$. The evaluation code uses β=1, fixed before evaluation and not tuned on a validation split. Section 6 reports the threshold-free AUROC and two operating points on held-out turns. The $1/\sqrt{H}$ factor makes $\bar{s}$ a per-coordinate root mean square. If a step difference $z \in \mathbb{R}^H$ has i.i.d. $\mathcal{N}(0, \sigma^2)$ coordinates, then $\|z\|_2/\sigma$ follows a χ_H distribution and

$$\mathbb{E}\|z\|_2 = \sigma\sqrt{2}\,\frac{\Gamma\left(\frac{H+1}{2}\right)}{\Gamma\left(\frac{H}{2}\right)} = \sigma\sqrt{H}\left(1 - \frac{1}{4H} + O(H^{-2})\right), \qquad (7)$$

so $\mathbb{E}\,\bar{s} \approx \sigma$ independently of H and V. The entropy term is unitless; the step term carries the units of h, and rescaling h by $c > 0$ is the same as replacing β by $c\beta$. This normalization therefore removes the dependence on H and V but not on the scale of h, so β has no natural value. Section 6 tests the score on held-out turns.

Algorithm 1 Tree-GNCA next-tool router (one turn). Training fits $\theta = (W_1, b_1, W_2, e_{1..K}, W_{\text{tool}}, b_{\text{tool}})$ by cross-entropy on the tool head; inference is deterministic.

Require: goal $\mathbf{g}$, last four steps $(u_{t-k}, r_{t-k})_{k=1}^{4}$; featuriser ϕ (TF-IDF/SVD fitted on training goals); tree operator A_{tree} (Eq. (2)); steps K; threshold θ

$o_0 \leftarrow [\phi(\mathbf{g});\ 4 \text{ trace scalars};\ \mathbf{0};\ 1]$ ▷ goal cell
for $i = 1, \ldots, 4$ **do** ▷ step cells, right-aligned: cell 4 is $t-1$
  $o_i \leftarrow [\phi(u\,r);\ s_i;\ \text{onehot}(u);\ 1]$ ▷ s_i: position, $\log|r|$, error, active
  (or $o_i \leftarrow \mathbf{0}$ if the step does not exist)
end for
$h_i^{(0)} \leftarrow \mathbf{0} \in \mathbb{R}^H$ for all cells
for $n = 0, \ldots, K-1$ **do** ▷ the same W_1, W_2 at every step and cell
  $x_i \leftarrow [o_i;\ h_i^{(n)} + e_n]; \quad p_i \leftarrow [x_i,\ (A\mathbf{x})_i,\ ((A-I)\mathbf{x})_i]$
  $[\delta_i; g_i] \leftarrow W_2\,\text{ReLU}(W_1 p_i + b_1); \quad h_i^{(n+1)} \leftarrow h_i^{(n)} + \sigma(g_i) \odot \delta_i$
end for
$\hat{y} \leftarrow \text{softmax}(W_{\text{tool}}\,\text{vec}(\mathbf{h}^{(K)}) + b_{\text{tool}})$
if $\mathcal{H}(\hat{y})/\ln|\mathcal{C}| > \theta$ **then** hand the turn to the large model **else return** $\arg\max \hat{y}$ ▷ entropy only; Sec. 6

4. Structural Properties & Formal Verification

The Lean 4 development (`formal_proofs/`, Lean 4.35.0, 28 top-level theorems) builds with `lake build` and contains no `sorry`. `#print axioms` shows that 13 theorems use no axioms and the rest use only Lean's standard axioms (`propext`, `Quot.sound`, `Classical.choice`); none depends on `sorryAx` or a user-declared axiom (Appendix A). Distances and weights in the Lean files are natural numbers, so the metric and contraction statements are about discrete abstractions of the real-valued model. Table 2 maps each group to what it does and does not say about the trained router; Appendix A lists every theorem by name.

Theorem 1 (Receptive-field separation; `theorem_receptive_field_separation`)**.** *Let $V \geq 3$. Every walk from cell 0 to cell $V-1$ in the chain graph has length at least $V-1$. In the star graph with hub 0, every ordered pair of cells is joined by a walk of length 2.*

With self-loops, information from cell j can influence cell i after K steps of Eq. (4) only if a walk of length $\leq K$ joins them, because each step mixes only graph neighbours. With V=5 and K=4 every topology therefore reaches every cell; the theorem explains why the goal reaches the history in fewer steps on the star and tree, not that they are more accurate. Companion theorems show that the chain, star and binary call-tree patterns (parent$(i) = \lfloor (i-1)/2 \rfloor$) are symmetric adjacency relations admitted by the model class. Neighbourhoods and walks commute with node permutations (`neighborhood_equivariance`, `path_equivariance`).

Theorem 2 (Tree simulation; `decision_tree_cellular_duality`)**.** *Let T be a binary decision tree with gates $g_u : \mathcal{X} \to \{0,1\}$ at internal nodes and labels at*

leaves. Let Φ_x replace a tree whose root has gate g and children (l, r) by l if $g(x) = 1$ and by r otherwise; leaves are fixed points. Then: (i) $\mathrm{eval}(\Phi_x(T), x) = \mathrm{eval}(T, x)$ *for every T and x. (ii) If* $\mathrm{depth}(T) \le K$*, then* $\Phi_x^K(T) = \mathrm{leaf}(\mathrm{eval}(T, x))$*. (iii) A complete tree of depth K has $2^K - 1 \ge K$ internal nodes. So M such trees with d parameters per split need $M(2^K - 1)d$ parameters, while one shared rule applied K times needs d.*

Proof sketch. (i) By cases on the root. For a leaf, Φ_x is the identity. For a branch, $\mathrm{eval}(T, x)$ is by definition eval of the child selected by $g(x)$, which is $\Phi_x(T)$. (ii) Induction on K. For K=0, $\mathrm{depth}(T) \le 0$ forces a leaf. For K+1, a leaf is fixed. A branch with depth $\le K$+1 steps to a child of depth $\le K$, and the induction hypothesis together with (i) gives the leaf $\mathrm{eval}(T, x)$. (iii) $2^K \ge K + 1$ by induction. □

This is a statement about an idealised discrete automaton: a tree-shaped cellular update *can* represent any depth-K decision tree with one shared rule, and K steps are enough. It does not say that gradient training finds such a rule, and the five-cell graph of Section 3.1 is not the decision tree itself. A further theorem (`sigmoidal_gating_*_margin_concordance`) states an elementary fact. A threshold at 1/2 on a sigmoid output agrees with the hard gate whenever the output is at least a margin $\delta > 0$ away from 1/2.

Proposition 3 (Exact structure of the implemented update)**.** *Let $\mathbf{h}^{(k)}(\mathbf{o})$ be the hidden states after k steps of Eq. (4) on a fixed operator A with self-loops, and $d_A(i, j)$ the hop distance in its pattern S. (i)* Weight tying. *The number of parameters is $N_0 + KH$, where N_0 does not depend on K. An untied K-layer network with the same perception and update has $N_0 + KH + (K-1)N_{upd}$, where $N_{upd} = D_{mid}(3D_{cell} + 1) + 2HD_{mid}$ is the size of one update network. (ii)* Locality. *$h_i^{(k)}$ is a function of $\{o_j : d_A(i, j) \le k\}$ only. (iii)* Automorphism equivariance. *If P is a permutation with $PAP^\top = A$, then $\mathbf{h}^{(k)}(P\mathbf{o}) = P\,\mathbf{h}^{(k)}(\mathbf{o})$ for every k and every parameter value. On A_{tree} the only non-trivial such P swaps cells 3 and 4 (t−2 and t−1). (iv)* Cost. *Up to element-wise terms, one turn costs $KV(6D_{cell}D_{mid} + 4HD_{mid}) + 2KV^2D_{cell} + 2VH|\mathcal{C}|$ floating-point operations, linear in K, with inference memory independent of K.*

Proof. (i) W_1, b_1, W_2 and the tool head are shared across steps; only e_n is per step. Untying adds one (W_1, b_1, W_2) per extra step. (ii) Induction on k. $h^{(0)} = 0$. Step k+1 of cell i reads x_i and $(A\mathbf{x})_i$, i.e. cells j with $S_{ij} = 1$, whose states depend on observations within k hops of j, hence within k+1 hops of i. (iii) $(AP\mathbf{x}) = P(A\mathbf{x})$ when $PAP^\top = A$; the cell-wise network and e_n are the same for every cell, so each step commutes with P, and so does the composition. For the tree, an automorphism must preserve degrees $(2, 3, 1, 1, 1$ for cells $0, \dots, 4)$ and adjacency, so it fixes cells 0 and 1 (unique degrees) and cell 2 (the only leaf adjacent to cell 0), and can only swap the two leaves under cell 1. (iv) Count the two 1×1 convolutions per cell and step, the aggregation $A\mathbf{x}$, and the readout. □

Parts (ii) and (iii) are exact for every parameter value, so they hold for the trained routers; a property test in the repository checks (i)–(iii) numerically on the model code, including that $h_i^{(k)}$ *does* change with o_j whenever $d_A(i, j) \le k$. Part (iii) means that the order of the two most recent steps reaches the tool head only through the position channel and the position-specific readout W_{tool}. For the default configuration (iv) gives $2.02\,M$ operations per turn.

Theorem 3 (Conditional contraction; `fixed_point_unique`, `trajectory_decay`)**.** *Let T be an operator on a metric space with $2\,d(Tx, Ty) \le d(x, y)$ for all x, y. Then T has at most one fixed point, and $d(T^n x, T^n y) \le 2^{-n} d(x, y)$. The premise holds whenever $b\,d(Tx, Ty) \le (r + \ell)\,d(x, y)$ for natural numbers with $2(r+\ell) \le b$ (`trained_damped_recurrence_is_dyadic_contraction`).*

For a damped update $h \leftarrow (1 - \rho)h + \rho F(h)$, r/b plays the role of the retention $1 - \rho$ and ℓ/b bounds the Lipschitz constant of ρF. The Lipschitz bound is a hypothesis. Appendix B derives the standard perturbation bound that would connect a contractive surrogate to a nearby operator. It reads $\limsup_n \|h^{(n)} - \tilde{h}^*\| \le \varepsilon/(1 - \tilde{L})$, where $\tilde{h}^*$ is the fixed point of an operator $\tilde{T}$ with Lipschitz constant $\tilde{L} < 1$ and $\sup_h \|T(h) - \tilde{T}(h)\| \le \varepsilon$. It too needs $\tilde{L} < 1$.

Empirical check of the contraction hypothesis. The Jacobian of one step of Eq. (4) with respect to the 5×16 hidden state is $J = I + \partial(\sigma(g) \odot \delta)/\partial h$; at initialisation $W_2 = 0$ and $J = I$. We computed J for every Hermes test turn, every step and every seed of the trained tree GNCA (42,360 Jacobians). Its spectral norm is 2.428 on average, 2.872 at the 95th percentile and 3.336 at most; 0.0% are below 1. The ratio of successive hidden-state update norms of the K=8 tree model averages 1.20, so updates grow rather than shrink. The trained router is not a contraction, and neither Theorem 3 nor the perturbation bound applies to it. We also trained a variant with spectrally normalised W_1, W_2 and a bounded update $h \leftarrow h + 0.25\,\sigma(g) \odot \tanh(\delta)$. It reaches only 61.99% Hermes test top-1 (68.90% for the unconstrained tree model in the same run; Table 6). Note that the identity term keeps the Lipschitz constant of this variant at up to $1 + 0.25L$, so spectral normalisation alone does not satisfy the premise either. With only K=4 steps the router is well defined without contraction, so the failed premise does not affect the results; it does rule out any certified stability claim.

5. Experimental Protocol

Data. Hermes: next-tool turns from public Hermes agent traces [30], split by session into 57,675 train-

Table 2: Scope of the formal results. "Proved" means machine-checked in Lean 4 for the stated (idealised, natural-number) objects. No row certifies the trained weights.

Group	Theorems	Proved	Relation to the trained router
Receptive fields	11	Chain needs $\geq V-1$ steps from goal to last cell; star joins all pairs in 2; chain, star and call tree are admissible patterns	Exact for the fixed graphs used; says nothing about accuracy (topologies within about 1 pp, Table 6)
Equivariance	3	Neighbourhoods and walks commute with relabelling	Holds for the message-passing step; the cell order is fixed by the trace, so it is not exploited
Tree simulation	5	K shared gated steps evaluate any decision tree of depth $\leq K$	Representability only; training is not shown to find such a rule
Contraction	5	Dyadic contraction $\Rightarrow$ unique fixed point and 2^{-n} decay; scaled Lipschitz bound $\Rightarrow$ dyadic	Premise fails: Jacobian norm 2.428 on average, 0.0% below 1
Aggregation, gating	4	Convex weights preserve bounds; sigmoid threshold matches the hard gate outside a margin	Elementary facts used in the design

ing, 3,465 validation and 3,530 test turns. BFCL: all public BFCL v3 single-turn AST, live and irrelevance files [27], with labels mapped heuristically onto the tool vocabulary. Rows with identical goal text stay in one split, stratified 60/20/20 by label (2,083/698/692). BFCL items are single-turn, so the history cells are mostly empty there. ToolBench [28] and API-Bank [29]: session-disjoint 60/20/20 splits (1,324/452/422 and 1,552/517/483). Split manifests and preparation scripts are in the repository.

Models. We compare the majority class, logistic regression, a Random Forest (50 trees, maximum depth 15) and HistGradientBoosting (200 iterations, 31 leaves, learning rate 0.1). The MLP has two hidden layers of 76 units (LayerNorm, GELU, dropout 0.1) on the flattened 5×146 cell features (62,713 parameters). The GNCA uses chain, star, tree and learned topologies. The primary tree baselines receive 188 features: the goal embedding, a positional one-hot of the last four tools and four scalars, but not the per-step result embeddings. To remove this confound we also fit both tree models on exactly the 730 flattened cell features of the MLP and GNCA (RF*, HGB*). They use the same hyperparameters and seeds (source S1, 111 s on CPU). These form a secondary comparison family. These are the default configurations; the next paragraph tunes all of them.

Hyperparameter search. Every learned model (logistic regression, RF, HGB, RF*, HGB*, MLP and the four GNCA topologies) is tuned separately on each benchmark with the same budget: 20 random-search trials per model and benchmark, 800 in total (search code in the repository). Trial 0 is the default configuration; the other trials are drawn with a fixed random seed per benchmark and model. Appendix D lists the search spaces and the selected configurations. Each trial is fitted on the training split with seed 0 and scored by validation top-1. The test split is not used during the search: neural trials receive the validation split in its place. The configuration with the highest validation score (on equal scores, the lower trial number) is refitted with seeds 0–2 and evaluated once on test. Fitting time was 1.6 h for the CPU models and 1.3 h for the GPU models (1.8 h wall clock, two workers in parallel on one machine). A host reboot interrupted the run once; it resumed from the per-trial logs and reran the two trials in flight. Thread counts affect results only through floating-point nondeterminism. Trial 0 reproduces the seed-0 validation score of the default run to within 1.16 pp. The exception is HistGradientBoosting on Hermes: the default run used scikit-learn's automatic early stopping (active above 10,000 training rows), while the search fixes early stopping off (difference up to 6.26 pp). The default won the search in 0 of the 40 model–benchmark pairs.

Main protocol: multi-seed selection and five seeds. The main comparison (Section 6.1, source S8) repeats the search with less selection noise. For each model and benchmark the 5 trials with the best seed-0 validation score are refitted with seeds 1 and 2. The configuration with the best three-seed mean validation top-1 is selected and refitted with 5 seeds (0–4) for testing. The test split is not used for selection.

New baselines. Three baselines that fit on the same GPU receive the same 20-trial budget and selection. k-NN retrieval [13] predicts from the nearest training turns ($k \in \{1, \ldots, 100\}$, uniform or distance weights, cosine or Euclidean metric, tabular or cell features). A pre-LayerNorm Transformer encoder [12] attends over the same five cells with a learned position embedding (width 32–128, 1–3 layers, 2 or 4 heads). The untied GNN has the tree GNCA's perception, gated update and readout, but separate update weights for each of the K steps; it isolates the effect of weight tying. Together with the ten earlier models there are 13 tuned models.

Held-out sessions and session-level proxy. An out-of-distribution split (S8, Table 4) holds 1,208 turns from 51 private Hermes sessions of one user. The code uses it for no training, featuriser fitting, search or selection. It is scored once with the final models; CIs resample sessions. Turns whose tool is outside the 13-tool

vocabulary are dropped. The majority-class floor predicts the most frequent tool of the Hermes test split. As an offline proxy for task success we report the share of test sessions in which every turn is routed correctly, with and without handing off the 20% least confident turns per seed.

Ablations. At the selected configuration of the tuned tree GNCA we change one component at a time (symmetric normalisation $Q^{-1/2}SQ^{-1/2}$, no $(A-I)x$ block, no gate, untied step weights) and fit five seeds per variant on every benchmark.

Training. Neural models use AdamW (learning rate 2×10^{-3}, weight decay 0.05, label smoothing 0.05, batch 512, cosine schedule) for 800 steps (3,000 on Hermes) and keep the checkpoint with the best validation top-1. Seeds 0, 1, 2 change the SVD fit, initialisation and batch order. Neural models were trained on one NVIDIA RTX 3060 and tree models on the CPU of the same machine, with Python 3.13.5, PyTorch 2.6.0 (CUDA 12.4), scikit-learn 1.9.1 and NumPy 2.5.4.

Statistics. The primary model is the tree GNCA with K=4 and label smoothing, compared with Random Forest, HistGradientBoosting and MLP on each benchmark (12 comparisons). For each comparison we average each test turn's correctness over the three seeds and bootstrap the mean paired difference over test turns (10,000 resamples). The resampling unit is thus the test turn, and n is the number of test turns, not seeds × turns. p-values are Holm-adjusted within the primary family (12) and the secondary family (8). Pooled effects use a DerSimonian–Laird random-effects model over the four benchmarks. Tuned models are compared in the same way, with three Holm families: tuned tree GNCA against tuned RF, HGB and MLP (12), against tuned RF* and HGB* (8), and tuned minus default for every model and benchmark (40). With 80% power at two-sided α=0.05 before Holm correction, the minimum detectable primary difference is 2.8 bootstrap standard errors, i.e. 0.9–4.1 pp. We test differences, not equivalence: a non-significant difference is reported as such. The main comparison (S8) uses the same bootstrap with five seeds averaged per turn and three Holm families: tree GNCA against RF, HGB and MLP (12), against RF* and HGB* (8), and against k-NN, Transformer and untied GNN (12). The ablations form one family of 16 and the OOD comparisons one family of 12 (session bootstrap). Escalation is scored by AUROC for detecting wrong routes, expected calibration error (15 bins) and accuracy on the most confident 80% of turns.

Hermes factorial. A separate Hermes run (source S2, 3 seeds, its own feature packing) varies depth $K\in\{2,4,8\}$, topology, the loss (label smoothing vs. class weights), cross-turn state carry, learned halting and the contractive variant. It also holds the escalation evaluation on the K=8 tree model. There β=1 and the thresholds $\theta\in\{0.50,0.65,0.76,0.85,1.00\}$ are fixed in the script, not tuned on a validation split; we report the threshold-free AUROC and two operating points on held-out test turns.

Latency. Batch size 1 on one thread of an AMD Ryzen 9 7950X 16-Core Processor (all thread pools set to one), 500 Hermes test turns after 30 warm-up turns, for every tuned model and seeds 0–2 (S8); we report medians. *Inference* is the model call alone: `predict_proba` for scikit-learn models and the PyTorch forward pass for neural models, both including framework overhead. *Featurisation* (TF-IDF, SVD and tabular or cell features computed from the raw turn) is timed separately as a preprocessing cost; *pipeline* is the sum. The earlier inference measurement for the default configurations (S1) uses 1,000 calls after 100 warm-up calls.

6. Results

6.1. Main comparison: tuned models, five seeds

Table 3 is the main result of this paper. Every model, including three new baselines (Section 5), was tuned per benchmark with the same budget, its configuration was chosen on the three-seed mean validation score, and it was refitted with five seeds. Multi-seed selection changed the chosen configuration in 25 of 52 model–benchmark pairs compared with seed-0 selection, so single-seed selection was noisy.

No model is best everywhere. HistGradientBoosting* is best on Hermes (70.41%), HistGradientBoosting on BFCL (78.70%), HistGradientBoosting* on ToolBench (77.16%) and the tree GNCA on API-Bank (87.16%). The tree GNCA reaches 69.31%, 77.17%, 76.35% and 87.16%.

Primary family. After Holm correction 1 of 12 comparisons is significant: the tree GNCA is ahead of the Random Forest on API-Bank (+3.98 pp, CI $[+1.57,+6.42]$, $p_{\text{Holm}}=0.014$). On Hermes the difference to the MLP is −0.58 pp (CI $[-1.13,-0.04]$); its CI excludes zero, but it does not survive Holm correction ($p_{\text{Holm}}=0.420$). Pooled, the GNCA differs from the Random Forest by +1.48 pp (CI $[-0.33,+3.30]$, $p=0.108$), from HistGradientBoosting by +0.10 pp (CI $[-1.05,+1.25]$) and from the MLP by +0.18 pp (CI $[-1.12,+1.48]$). None of these is significant. We test differences, not equivalence, so this does not show that the models are equally accurate.

Input-matched trees. HistGradientBoosting on the GNCA's own cell features (HGB*) is ahead of the GNCA on Hermes (−1.10 pp, CI $[-1.89,-0.32]$, $p_{\text{Holm}}=0.036$), the largest dataset. The GNCA is ahead of RF* on API-Bank (+3.93 pp, $p_{\text{Holm}}=0.007$). The pooled differences are +1.29 pp (CI $[-0.85,+3.43]$) against RF*

Table 3: Main result. Test top-1 accuracy (%, mean ± SD over five seeds) of every model after the equal-budget search with multi-seed selection, and the paired difference tree GNCA minus model (Δ, pp; seed-clustered bootstrap over test turns; †: Holm-adjusted $p < 0.05$ within the model's family). Pooled: DerSimonian–Laird random-effects estimate over the four benchmarks with 95% CI. Families: primary (RF, HGB, MLP; 12 tests), secondary (RF*, HGB*; 8), new baselines (k-NN, Transformer, untied GNN; 12). Source S8 (Table 12).

	Hermes		BFCL		ToolBench		API-Bank		Pooled Δ
Model	Top-1	Δ	Top-1	Δ	Top-1	Δ	Top-1	Δ	[95% CI]
Logistic regression	65.60 ± 0.09	–	76.71 ± 0.41	–	64.60 ± 1.36	–	85.18 ± 0.73	–	–
Random Forest	69.16 ± 0.08	+0.16	75.14 ± 0.27	+2.02	76.02 ± 0.35	+0.33	83.19 ± 0.45	+3.98†	+1.48 [-0.33, +3.30]
HistGradientBoosting	69.36 ± 0.19	-0.05	78.70 ± 0.22	-1.53	75.97 ± 0.85	+0.38	85.47 ± 0.64	+1.70	+0.10 [-1.05, +1.25]
Random Forest*	69.92 ± 0.28	-0.61	75.29 ± 0.23	+1.88	75.83 ± 1.10	+0.52	83.23 ± 0.33	+3.93†	+1.29 [-0.85, +3.43]
HistGradientBoosting*	70.41 ± 0.37	-1.10†	78.03 ± 0.45	-0.87	77.16 ± 0.68	-0.81	85.51 ± 0.77	+1.66	-0.43 [-1.62, +0.76]
k-NN retrieval	66.87 ± 0.08	+2.44†	76.27 ± 0.43	+0.90	75.97 ± 0.32	+0.38	82.48 ± 0.50	+4.68†	+2.21 [+0.65, +3.78]
MLP	69.90 ± 0.37	-0.58	78.50 ± 0.69	-1.33	74.50 ± 0.57	+1.85	85.88 ± 0.63	+1.28	+0.18 [-1.12, +1.48]
Transformer	69.93 ± 0.28	-0.62	76.85 ± 0.50	+0.32	75.59 ± 0.56	+0.76	85.42 ± 1.44	+1.74	+0.47 [-0.67, +1.60]
GNN (untied steps)	69.18 ± 0.36	+0.13	76.88 ± 0.79	+0.29	75.45 ± 1.26	+0.90	86.21 ± 1.33	+0.95	+0.38 [-0.03, +0.79]
GNCA chain	69.66 ± 0.42	–	76.76 ± 1.20	–	76.11 ± 0.51	–	86.67 ± 1.15	–	–
GNCA star	69.31 ± 0.51	–	76.97 ± 0.30	–	75.26 ± 1.45	–	86.50 ± 0.47	–	–
GNCA learned	69.52 ± 0.23	–	76.27 ± 0.63	–	75.59 ± 1.10	–	86.38 ± 1.18	–	–
GNCA tree (primary)	69.31 ± 0.13	–	77.17 ± 1.14	–	76.35 ± 1.29	–	87.16 ± 0.84	–	–

and −0.43 pp (CI [−1.62, +0.76]) against HGB*.

New baselines. The GNCA is ahead of k-NN retrieval (pooled +2.21 pp, CI [+0.65, +3.78], $p = 0.005$; significant after Holm on Hermes and API-Bank). It shows no significant difference from the small Transformer over the same five cells (+0.47 pp, CI [−0.67, +1.60]) or from the untied GNN with the same perception and update (+0.38 pp, CI [−0.03, +0.79], $p = 0.068$). With five seeds the minimum detectable primary difference is 0.6–4.4 pp, depending on the benchmark. The tuned tree GNCAs have 108,974–150,062 parameters, the tuned Transformers 47,501–622,861, the untied GNNs 94,622–408,014 and the MLPs 62,713–320,125.

With matched tuning and five seeds the tree GNCA is a competitive small router, not a more accurate one. Its pooled differences to gradient boosting, the MLP, the Transformer and the untied GNN are not significant. It is ahead of k-NN retrieval and of the Random Forest on API-Bank, and behind gradient boosting on cell features on Hermes.

6.2. Held-out sessions, session-level routing and inference latency

Out of distribution. On the held-out private sessions (Table 4, Figure 2b) every model loses 11.76–14.40 pp against its Hermes test accuracy. The tree GNCA falls from 69.31% to 56.61% (CI [48.57, 63.54]). The majority-class rate is 49.17%, and 9 of 13 models have a session-bootstrap CI above it. Paired with the tree GNCA, only the chain GNCA differs after Holm correction (+1.23 pp in favour of the tree, $p_{\text{Holm}} = 0.026$); the other differences are at most 3.82 pp and not significant. The drop is shared by all models, so it reflects the shift between public and private traces, not a weakness specific to the GNCA. With one user and 51 sessions the CIs are wide.

Session-level routing. We do not execute agents, so we cannot measure task success. As an offline proxy we count Hermes test sessions (on average 18.5 turns) in which every turn is routed correctly. This share is low for every model: 3.8–5.7% (tree GNCA 5.0%). If the 20% least confident turns are handed to a fallback and counted as correct, an optimistic assumption, the share rises to 7.3–11.3% (tree GNCA 11.0%). On the OOD sessions it is 0.8–5.9%. ToolBench and API-Bank test sessions contain one turn each and BFCL is single-turn, so the measure is not defined there. A router at this accuracy cannot run a multi-step session unattended; it needs the fallback.

Inference latency. On one CPU thread the tuned tree GNCA's inference takes 0.559 ms per turn (95th percentile 0.676 ms; Table 4, Figure 2a). The neural models take 0.238–1.381 ms: the MLP 0.238, the untied GNN 0.531 and the Transformer 0.885 ms. Logistic regression is fastest (0.102 ms) and least accurate. The tree ensembles take 4.442 ms (Random Forest) and 9.781 ms (HistGradientBoosting), and k-NN retrieval, which searches the whole training set, 51.276 ms. Every model's inference is far below the 378 and 848 ms per decision of the local LLMs in Section 1.

Featurisation cost. Before inference, every model needs its input features. In our unoptimised Python pipeline this preprocessing takes 3.27–3.88 ms per turn for the tabular features of logistic regression and the primary tree models, and 7.46–9.02 ms for the cell features used by the GNCA, the MLP, the Transformer, the untied GNN, RF*, HGB* and k-NN (7.65 ms for the tree GNCA). It is shared by every feature-based model and is not part of the router. Including it, a routing decision takes 8.19 ms for the tree GNCA, 7.82 ms for the

Table 4: Deployment-oriented measures for the tuned models (five seeds). OOD: 1,208 turns from 51 private Hermes sessions of one user, never used for training, selection or tuning; 95% CI by session bootstrap; Δ: tree GNCA minus model (†: Holm $p < 0.05$ over 12). Sessions all correct: share of sessions in which every turn is routed correctly (Hermes test, 191 sessions; OOD, 51). CPU time: batch 1, one thread, median over 500 Hermes test turns, mean over seeds 0–2; inference is the model call alone, featurisation (TF-IDF, SVD and tabular or cell features, unoptimised Python) is preprocessing, pipeline is their sum. Size: serialised model without the shared featuriser. Source S8.

	OOD (held-out sessions)		Sessions all correct (%)		CPU time per turn, 1 thread (ms, median)			Size
Model	Top-1 [95% CI]	Δ vs. GNCA	Hermes test	OOD	Inference	Featurisation	Pipeline	kB
Logistic regression	52.78 [46.1, 60.2]	+3.82	3.8	5.9	0.102	3.27	3.37	10
Random Forest	56.77 [48.0, 63.8]	-0.17	5.3	2.0	4.442	3.37	7.82	154788
HistGradientBoosting	55.43 [47.0, 62.5]	+1.18	5.5	2.0	9.781	3.88	13.67	25456
Random Forest*	57.65 [49.9, 64.1]	-1.04	5.2	2.0	1.380	7.46	8.85	72768
HistGradientBoosting*	57.24 [48.4, 64.4]	-0.63	5.1	1.6	5.151	8.42	13.57	8541
k-NN retrieval	55.12 [45.4, 62.9]	+1.49	4.8	0.8	51.276	9.02	60.24	164915
MLP	56.71 [48.9, 63.3]	-0.10	5.7	1.6	0.238	7.92	8.15	248
Transformer	55.53 [47.1, 62.3]	+1.08	5.4	0.8	0.885	8.43	9.34	2447
GNN (untied steps)	56.72 [48.6, 63.6]	-0.12	5.3	2.0	0.531	8.76	9.30	376
GNCA chain	55.38 [46.7, 62.3]	+1.23†	5.1	0.8	1.307	7.59	8.89	373
GNCA star	55.98 [48.1, 62.8]	+0.63	5.0	2.4	1.381	8.05	9.42	429
GNCA learned	56.13 [47.6, 62.9]	+0.48	5.4	2.0	0.790	8.56	9.34	365
GNCA tree (primary)	56.61 [48.6, 63.5]	–	5.0	3.5	0.559	7.65	8.19	589
Majority class	49.17 [39.6, 58.9]							

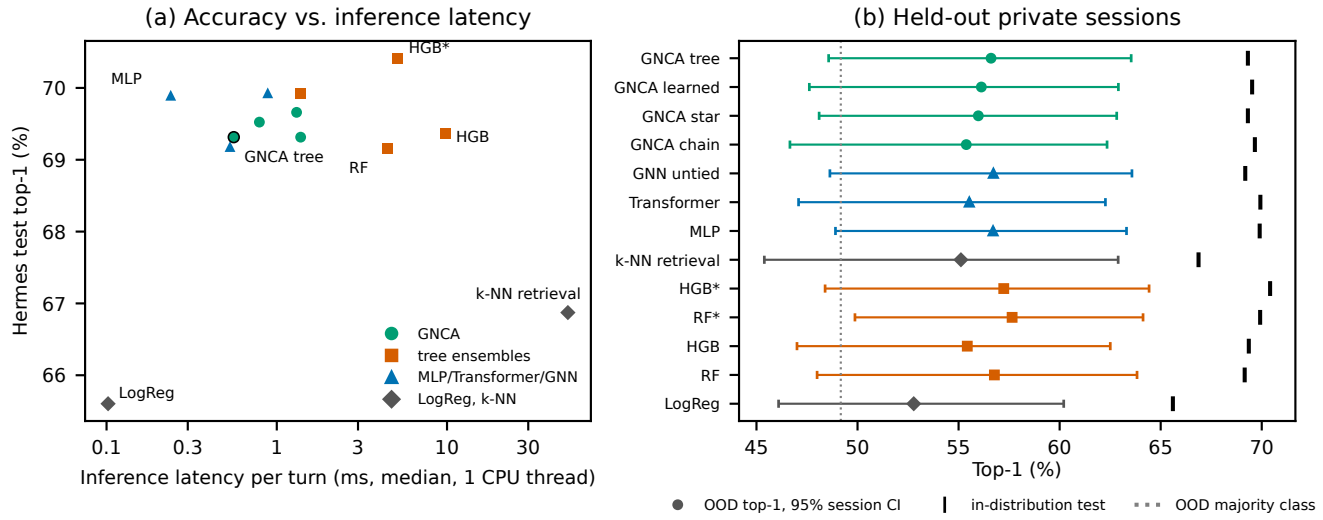


Figure 2: (a) Hermes test top-1 against CPU inference latency per turn (model call only, featurisation excluded and reported in Table 4; tuned models, five seeds; marker shape and colour give the model family, the outlined circle is the tree GNCA; unlabelled points are the other GNCA topologies, RF* and the neural baselines in Table 4). (b) Top-1 on held-out private sessions (dots, 95% session-bootstrap CI) and on the in-distribution Hermes test split (bars). Source S8.

Random Forest and 13.67 ms for HistGradientBoosting, so in this pipeline the featuriser, not the model, sets the per-turn cost.

6.3. Component ablations and escalation

Ablations. Changing one component of the tuned tree GNCA at a fixed configuration (Table 5) changes pooled test top-1 by −0.37 pp for symmetric instead of row normalisation (CI $[-0.89, +0.15]$), −0.08 pp without the $(A-I)x$ block, −0.01 pp without the gate and −0.13 pp with untied step weights (CI $[-0.58, +0.32]$). 0 of the 16 per-benchmark differences are significant after Holm. The $(A-I)x$ block is linearly redundant given x and Ax (Section 3.3), so its null effect is expected. Weight tying saves $(K-1)N_{\text{upd}}$ parameters (Proposition 3) without a detectable loss of accuracy; it does not improve accuracy. We found no accuracy effect of the normalisation or the gate at this scale.

Table 5: Upper block: one-component ablations of the tuned tree GNCA at its selected configuration (difference in test top-1, pp, five seeds; Holm over 16; none significant). Lower block: escalation and calibration of the tuned tree GNCA (five seeds). Source S8.

	Hermes	BFCL	ToolBench	API-Bank	Pooled [95% CI]
Ablation: change in top-1 (pp) vs. the tuned tree GNCA, same configuration					
symmetric normalisation	+0.06	-0.87	-0.95	-0.12	-0.37 [-0.89, +0.15]
no $(A-I)x$ block	+0.01	+0.32	-0.95	+0.08	-0.08 [-0.47, +0.32]
no gate	+0.02	+0.17	-0.47	+0.21	-0.01 [-0.31, +0.28]
untied weights	-0.32	-0.87	+0.00	+0.46	-0.13 [-0.58, +0.32]
Escalation of the tuned tree GNCA (5 seeds, mean ± SD)					
AUROC, entropy	0.740 ± 0.005	0.697 ± 0.030	0.678 ± 0.024	0.841 ± 0.013	
AUROC, s_{esc} (β=1)	0.725 ± 0.006	0.648 ± 0.048	0.696 ± 0.025	0.669 ± 0.032	
AUROC, step norm only	0.451 ± 0.028	0.413 ± 0.047	0.560 ± 0.012	0.447 ± 0.067	
ECE (%)	3.46 ± 0.44	5.08 ± 2.31	12.20 ± 0.83	6.72 ± 0.90	
Top-1 on 80% most confident	75.81 ± 0.14	82.20 ± 0.75	83.14 ± 1.98	94.87 ± 0.79	

Escalation. For the tuned tree GNCA, predictive entropy separates wrong from correct routes with AUROC 0.740 (Hermes), 0.697 (BFCL), 0.678 (ToolBench) and 0.841 (API-Bank). Adding the step-norm term (β=1) lowers the AUROC on 3 of 4 benchmarks, and the step norm alone is below 0.5 on 3. This confirms the earlier finding on the K=8 model (S2) for the primary model: entropy alone should be used. Keeping the 80% most confident turns raises top-1 from 69.31 to 75.81% on Hermes and from 87.16 to 94.87% on API-Bank. Entropy is not more informative for the GNCA than for the baselines. HistGradientBoosting reaches AUROC 0.872 on ToolBench and 0.761 on BFCL, but its expected calibration error there is 18.9% and 17.4%, against 12.2% and 5.1% for the GNCA.

6.4. Earlier protocols, depth and rare tools

The main comparison was preceded by two three-seed protocols, reported in full in Appendix C: default configurations (Tables 7–8) and a single-seed search (Tables 10–11). With default settings the tree GNCA was ahead of the Random Forest pooled (+2.55 pp, CI [+0.67, +4.43]). After the single-seed search that lead fell to +1.99 pp (CI [−0.30, +4.27]), and in the main comparison it is +1.48 pp, not significant. Which model wins on a single benchmark changed with tuning, selection and the features given to the tree models. The pooled differences to HistGradientBoosting and the MLP were not significant under all three protocols (+0.75, −0.11 and +0.10 pp against HistGradientBoosting).

Depth, topology and loss. In the factorial run (Table 6) going from K=2 to K=4 helps (67.40% → 68.90%) and K=8 adds nothing (68.92%). The topologies span about one percentage point, from 67.88% (chain) to 68.90% (tree), with the star at 68.77% and the learned graph at 68.03%. This is consistent with Theorem 1: with K=4 every topology reaches all cells. Cross-turn state carry and learned halting do not help. On the learning curve, the best-so-far validation accuracy of the tree model (K=4) is 68.15% after 500 steps and 70.79% after 3,000. The gains at the intermediate checkpoints are gradual, with no sudden late jump. We therefore see no delayed-generalisation ("grokking") phase and dropped that term from the title.

Table 6: Hermes factorial (test top-1, %, mean ± SD over 3 seeds; source S2). Columns: training loss, label smoothing (LS) or class weights (CW). Carry: hidden state carried across turns; halting: learned halting. Contractive variant: spectrally normalised W_1, W_2, update $0.25\,\sigma(g) \odot \tanh(\delta)$.

Configuration	Params	LS	CW
Tree, K=2	59,150	67.40 ± 0.20	63.81 ± 0.71
Tree, K=4	59,182	68.90 ± 0.21	64.47 ± 0.73
Tree, K=8	59,246	68.92 ± 0.47	64.33 ± 0.18
Chain, K=4	59,182	67.88 ± 0.23	64.70 ± 0.34
Star, K=4	59,182	68.77 ± 0.24	64.63 ± 0.12
Learned, K=4	59,207	68.03 ± 0.36	64.35 ± 0.88
Tree, K=4, carry	59,182	67.55 ± 0.07	62.93 ± 0.78
Tree, K=4, halting	59,199	67.12 ± 0.26	63.41 ± 0.54
Tree, K=4, contractive	59,182	61.99 ± 0.14	63.23 ± 0.31

Rare tools. Class-weighted training trades top-1 for macro-F1. On Hermes (Table 7 run) the class-weighted tree GNCA has macro-F1 43.02% at 65.46% top-1, against 35.08% at 69.10% with label smoothing. Random Forest and HistGradientBoosting reach 28.46% and 31.56% macro-F1, the MLP 39.74%.

Inference latency, default configurations. Timing the model call alone (Section 5), the default tree GNCA takes 0.403 ms (95th percentile 0.461 ms), HistGradientBoosting 0.532 ms and the Random Forest 1.209 ms. Serialised sizes are 240 kB (GNCA weights), 0.78 MB (HistGradientBoosting) and 51.62 MB (Random Forest, 307,118 nodes). Like the main measurement, these inference times exclude featurisation (Section 6.2).

Escalation, K=8 model. In the factorial run (S2, 3 seeds, Hermes) s_{esc} reaches AUROC 0.730 ± 0.008, entropy alone 0.732 ± 0.001 and the step norm alone 0.441 ± 0.023. At the fixed thresholds θ=0.76 and 0.50 the router escalates 11.4% and 72.6% of turns and is 72.45% and 90.48% accurate on the turns it keeps. The

pattern is the same as for the tuned primary model (Section 6.3).

7. Discussion and Limitations

Where the GNCA fits. With equal tuning, multi-seed selection and five seeds, the tree GNCA shows no significant pooled difference from gradient boosting, a same-size MLP, a small Transformer or an untied GNN (Table 3). It is ahead of k-NN retrieval. Gradient boosting on the GNCA's own cell features is ahead of it on Hermes, the largest benchmark. None of the parts that distinguish the GNCA (weight tying, gate, the $(A-I)x$ block, the normalisation, the step-norm uncertainty) changed accuracy or error detection in our runs (Table 5). What tying does buy is a parameter count that grows by only H per step, exact locality and equivariance (Proposition 3), and a model that is differentiable end to end. We regard this as an early-stage result: a new architecture that matches mature baselines on its first careful evaluation, with no evidence yet that it beats them.

Limitations.

- **Novelty.** The cell rule is a standard GNCA update and a member of the weight-tied message-passing family (Section 2). What is new is the router (Algorithm 1), its structural properties and the controlled evaluation, not a new primitive. The ablations show that its specific components are not what makes it accurate.
- **Search budget.** Each model received 20 random-search trials per benchmark; only the top 5 were re-scored with three seeds. A larger budget could change the ranking. The HistGradientBoosting search did not include early stopping. Tuned model sizes are not matched (Section 6.1).
- **Small test sets.** ToolBench, API-Bank and BFCL have 422–692 test turns. Even with five seeds, differences below about 4.4 pp can go undetected there, and seeds do not add test turns. Non-significant differences are not evidence of equivalence.
- **Baselines not run.** The two local LLMs ran zero-shot without tool-specific prompting or fine-tuning; they show cost, not the best LLM accuracy. We ran no hosted LLM, no fine-tuned small function-calling model [22–24] and no learned tool retriever [25]. A protocol for a hosted-LLM baseline on the same splits is in the repository; it was not run because it needs a paid API.
- **Turn-level evaluation and an offline proxy.** We do not run agents. The session measure of Section 6.2 only counts sessions without routing errors; it ignores whether a wrong tool would have been recovered from, and it assumes that handed-off turns are resolved. It is defined only on Hermes.
- **Out-of-distribution test on one user.** The held-out sessions come from one user and 51 sessions, so the CIs are wide and other shifts (other harnesses, new tools) are untested.
- **Escalation.** The weight β=1 was fixed, not tuned, and the entropy hand-off threshold was not chosen on validation; we report AUROC and a fixed 80% coverage point.
- **Fixed vocabulary and window.** Every benchmark is mapped onto 13 tools and a five-cell window. New tools (e.g. dynamically loaded MCP servers) need a new head or an open-vocabulary scorer; longer histories need pooling or more cells. We did not test either, nor other window lengths or cell features.
- **Formal results are idealised.** The Lean results concern idealised automata, and nothing is proved about trained weights; the trained recurrence is not contractive. Proposition 3(ii)–(iii) hold for any weights but are about structure, not accuracy.
- **Reproducibility on GPU.** Refitting the Hermes models from the stored configurations (refit check, S8) reproduces the reported test accuracy exactly in 34 of 39 model–seed pairs. The largest difference, 1.05 pp (GNCA chain), comes from floating-point nondeterminism on the GPU. CPU models reproduce exactly. Latency was measured on one machine, and the featuriser is unoptimised Python.

Broader impact. A cheap router reduces the energy spent on routine turns, but faster loops can also propagate agent errors faster. Harnesses should keep permission sandboxes and human approval for irreversible actions regardless of the router.

8. Conclusion

A graph cellular automaton over five trace cells is a competitive next-tool router. We tuned it with the same budget as the other models, selected every model on multi-seed validation and tested with five seeds on four benchmarks. It shows no significant pooled difference from gradient boosting, an MLP, a Transformer or an untied GNN, and it is ahead of k-NN retrieval. Gradient boosting on its own features is ahead of it on Hermes. Its distinctive components have no detectable effect on accuracy, every model loses about the same amount on held-out private sessions, and inference takes well under a millisecond on one CPU thread, while our unoptimised featuriser dominates the per-turn pipeline cost. The diffusion view and the structural proposition explain what the architecture computes; the Lean results describe what it can represent, not what the trained model does. Useful next steps are an optimised featuriser, larger test sets, out-of-distribution data from more users, and an agent-level evaluation that measures task success.

Data and code availability. Code, split manifests, result files, the table and figure generators and the Lean proofs: `https://github.com/cap-jmk-real/nca_research` (MIT licence). Every number in this paper is a macro written by the four table generators in `script`

s/ from the result files listed in Table 12. Checkpoints too large for git are listed with SHA-256 hashes in the artifact manifest in stage0/stage0_results/e1v2/.

Use of AI tools. AI coding and writing assistants were used to write code, run experiments and draft text. The author checked the result files and is responsible for the content.

Competing interests and funding. None.

References

[1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin. Growing neural cellular automata. *Distill*, 5(2):e23, 2020. https://doi.org/10.23915/distill.00023

[2] D. Grattarola, L. Livi, and C. Alippi. Learning graph cellular automata. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2110.14237.

[3] G. Gala, D. Grattarola, and E. Quaeghebeur. E(n)-equivariant graph neural cellular automata. arXiv:2301.10497, 2023.

[4] M. Etcheverry et al. Reasoning with neural cellular automata. arXiv:2609.36126, 2026.

[5] L. Pio-Lopez et al. BraiNCA: brain-inspired neural cellular automata and applications to morphogenesis and motor control. arXiv:2604.01932, 2026.

[6] A. Mittal et al. TextNCA: Neural cellular automata for language modeling via hierarchical local attention. arXiv:2608.02050, 2026.

[7] D. Lee et al. Training language models via neural cellular automata. arXiv:2603.10055, 2026.

[8] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel. Gated graph sequence neural networks. In *Proc. ICLR*, 2016. arXiv:1511.05493.

[9] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. Neural message passing for quantum chemistry. In *Proc. ICML*, 2017. arXiv:1704.01212.

[10] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser. Universal transformers. In *Proc. ICLR*, 2019. arXiv:1807.03819.

[11] S. Bai, J. Z. Kolter, and V. Koltun. Deep equilibrium models. In *NeurIPS*, 2019. arXiv:1909.01377.

[12] A. Vaswani et al. Attention is all you need. In *NeurIPS*, 2017. arXiv:1706.03762.

[13] T. M. Cover and P. E. Hart. Nearest neighbor pattern classification. *IEEE Trans. Inf. Theory*, 13(1):21–27, 1967. https://doi.org/10.1109/TIT.1967.1053964

[14] Y. Geifman and R. El-Yaniv. Selective classification for deep neural networks. In *NeurIPS*, 2017. arXiv:1705.08500.

[15] P. Kontschieder, M. Fiterau, A. Criminisi, and S. Rota Bulò. Deep neural decision forests. In *Proc. ICCV*, 2015. https://doi.org/10.1109/ICCV.2015.172

[16] N. Frosst and G. Hinton. Distilling a neural network into a soft decision tree. arXiv:1711.09784, 2017.

[17] Y. Yang, I. G. Morillo, and T. M. Hospedales. Deep neural decision trees. arXiv:1806.06988, 2018.

[18] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176, 2023.

[19] I. Ong et al. RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665, 2024.

[20] S. Singla et al. AgentWeave: Routing before reasoning for efficient function calling in tool-rich language models. arXiv:2608.23078, 2026.

[21] J. Li. Fast models, slow evidence: A paired and self-audited evaluation of System-1 decision models for LLM agent harnesses. arXiv:2610.02267, 2026.

[22] W. Chen and Z. Li. Octopus v2: On-device language model for super agent. arXiv:2404.01744, 2024.

[23] L. E. Erdogan et al. TinyAgent: Function calling at the edge. arXiv:2409.00608, 2024.

[24] Q. Lin et al. Hammer: Robust function-calling for on-device language models via function masking. arXiv:2410.04587, 2024.

[25] C. Qu et al. Towards completeness-oriented tool retrieval for large language models. arXiv:2405.16089, 2024.

[26] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Gorilla: Large language model connected with massive APIs. arXiv:2305.15334, 2023.

[27] S. G. Patil et al. The Berkeley Function Calling Leaderboard (BFCL), dataset. https://huggingface.co/datasets/gorilla-llm/Berkeley-Function-Calling-Leaderboard, 2024.

[28] Y. Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *Proc. ICLR*, 2024. arXiv:2307.16789.

[29] M. Li et al. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proc. EMNLP*, 2023. arXiv:2304.08244.

[30] Hermes agent reasoning traces (filtered). https://huggingface.co/datasets/DJLougen/hermes-agent-traces-filtered, derived from https://huggingface.co/datasets/lambda/hermes-agent-reasoning-traces, accessed 2026.

[31] F. R. K. Chung. *Spectral Graph Theory*. CBMS Regional Conference Series in Mathematics 92, American Mathematical Society, 1997. https://doi.org/10.1090/cbms/092

[32] G. Katz et al. The Marabou framework for verification and analysis of deep neural networks. In *Proc. CAV*, LNCS 11561, pp. 443–452, 2019. https://doi.org/10.1007/978-3-030-25540-4_26

[33] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter. Beta-CROWN: Efficient bound propagation with per-neuron split constraints for complete and incomplete neural network robustness verification. In *NeurIPS*, 2021. arXiv:2103.06624.

A. Lean Axiom Audit

Output of `#print axioms` for every top-level theorem (`stage0/stage0_results/review_1110/lean_axioms.txt`), generated with `lake env lean` in `formal_proofs/`. "Standard" axioms are Lean's `propext`, `Quot.sound` and `Classical.choice`.

```
'NCAMetaLearning.fixed_point_unique' depends on axioms: [propext, Classical.choice, Quot.sound]
'NCAMetaLearning.trajectory_decay' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.cellularTreeStep_preserves_eval' does not depend on any axioms
'NCAMetaLearning.iterate_cellular_tree_preserves_eval' does not depend on any axioms
'NCAMetaLearning.cellular_tree_converges_in_depth' depends on axioms: [propext, Classical.choice, Quot.sound]
'NCAMetaLearning.tree_automaton_edges_in_gnca' does not depend on any axioms
'NCAMetaLearning.powTwo_ge_succ' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.decision_tree_cellular_duality' depends on axioms: [propext, Classical.choice, Quot.sound]
'NCAMetaLearning.permuteAdj_symm' does not depend on any axioms
'NCAMetaLearning.neighborhood_equivariance' does not depend on any axioms
'NCAMetaLearning.path_equivariance' does not depend on any axioms
'NCAMetaLearning.lineGraphAdj_symm' does not depend on any axioms
'NCAMetaLearning.starGraphAdj_symm' does not depend on any axioms
'NCAMetaLearning.theorem_subsumption_1d' does not depend on any axioms
'NCAMetaLearning.star_graph_diameter_two' does not depend on any axioms
'NCAMetaLearning.line_graph_distance_bound' depends on axioms: [propext, Classical.choice, Quot.sound]
'NCAMetaLearning.theorem_receptive_field_separation' depends on axioms: [propext, Classical.choice, Quot.sound]
'NCAMetaLearning.treeGraphAdj_symm' does not depend on any axioms
'NCAMetaLearning.theorem_tree_subsumption' does not depend on any axioms
'NCAMetaLearning.dyadic_of_scaled_lip' depends on axioms: [propext]
'NCAMetaLearning.trained_damped_recurrence_is_dyadic_contraction' depends on axioms: [propext]
'NCAMetaLearning.trained_damped_recurrence_perturbation_decay' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.weighted_sum_bounded_two' depends on axioms: [propext]
'NCAMetaLearning.weighted_sum_bounded_three' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.call_tree_parent_is_ancestor' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.call_tree_parent_adj' does not depend on any axioms
'NCAMetaLearning.sigmoidal_gating_positive_margin_concordance' depends on axioms: [propext, Quot.sound]
'NCAMetaLearning.sigmoidal_gating_negative_margin_concordance' depends on axioms: [propext, Quot.sound]
Lean (version 4.35.0, x86_64-unknown-linux-gnu, commit eb6cafb7290e8bd5f2937522d8d804f58f02bafc, Release)
```

Theorem 2 is `decision_tree_cellular_duality` in the directory `formal_proofs/`.

B. Perturbation Bound

Proposition 4. *Let $\tilde{T}$ be $\tilde{L}$-Lipschitz on a complete normed space with $\tilde{L} < 1$ and fixed point $\tilde{h}^*$, and let T satisfy $\|T(h) - \tilde{T}(h)\| \le \varepsilon$ for all h. Then the iterates $h^{(n+1)} = T(h^{(n)})$ satisfy*

$$\|h^{(n)} - \tilde{h}^*\| \le \tilde{L}^n \|h^{(0)} - \tilde{h}^*\| + \varepsilon \frac{1 - \tilde{L}^n}{1 - \tilde{L}},$$

$$\limsup_{n\to\infty} \|h^{(n)} - \tilde{h}^*\| \le \frac{\varepsilon}{1 - \tilde{L}}.$$

Proof. Let $d_n = \|h^{(n)} - \tilde{h}^*\|$. Since $\tilde{h}^* = \tilde{T}(\tilde{h}^*)$, $d_{n+1} \le \|T(h^{(n)}) - \tilde{T}(h^{(n)})\| + \|\tilde{T}(h^{(n)}) - \tilde{T}(\tilde{h}^*)\| \le \varepsilon + \tilde{L} d_n$. Unrolling gives $d_n \le \tilde{L}^n d_0 + \varepsilon \sum_{m=0}^{n-1} \tilde{L}^m$, which is the bound for finite n; letting $n \to \infty$ with $\tilde{L}^n \to 0$ gives the limit. □

For the GNCA, T is one step of Eq. (4). The bound would apply if a contractive surrogate $\tilde{T}$ were found within ε of the trained step. Section 4 shows that the trained step itself has Jacobian norm above one at every sampled state; we know of no such surrogate and do not claim one.

C. Default Configurations and the Single-Seed Search

Accuracy. These three-seed runs preceded the main comparison (Section 6.1): default configurations (Tables 7–8, Figures 3–4) and the single-seed search (Tables 10–11, Figure 5). We keep them because they show how much the conclusions depend on tuning and selection. Table 7 and Figure 3 give all models on all benchmarks. Every learned model is far above the majority class. The tree GNCA reaches 69.10% on Hermes, 76.40% on BFCL, 76.30% on ToolBench and 85.99% on API-Bank. No model wins everywhere: HistGradientBoosting is best on BFCL (78.18%), the MLP on Hermes (69.91%), and the star GNCA on API-Bank (87.09%). Choosing the GNCA topology on validation instead of fixing the tree would pick the star on Hermes (69.66% test) and the chain on BFCL (76.01%). It would pick the learned topology on ToolBench (75.75%) and the star on API-Bank (87.09%). We keep the pre-specified tree for all tests.

Paired tests. After Holm correction (Table 8, Figure 4) two of twelve primary comparisons are significant at 0.05. With default settings the tree GNCA is ahead of HistGradientBoosting on Hermes (+2.90 pp, $p_{\text{Holm}} = 0.001$) and the Random Forest on API-Bank (+5.87 pp, $p_{\text{Holm}} = 0.002$). On BFCL its difference to HistGradientBoosting is −1.78 pp (CI $[-3.71, +0.14]$, not significant). Pooled over benchmarks, the GNCA is ahead of the Random Forest (+2.55 pp, CI $[+0.67, +4.43]$, $p = 0.008$). It shows no significant difference from HistGradientBoosting (+0.75 pp, between-benchmark variance $\tau^2 = 4.36$) or the MLP (−0.26 pp, CI $[-1.43, +0.92]$).

Input-matched tree models. Given the same cell features (lower block), the Random Forest closes the gap on Hermes (+0.07 pp, CI $[-0.81, +0.95]$) but remains below the GNCA on the other three benchmarks (significant after Holm on BFCL and API-Bank). Pooled, the GNCA is ahead by +2.60 pp (CI $[+0.12, +5.07]$, $p = 0.040$). Against HistGradientBoosting on cell features (Table 8) the pooled difference is +0.12 pp (CI $[-2.02, +2.25]$). The GNCA is ahead on Hermes (+2.29 pp, $p_{\text{Holm}} = 0.001$), with no significant difference on the other three. On BFCL the history cells are mostly empty. HistGradientBoosting reaches the same accuracy with either feature set, while the Random Forest, which

Table 7: Test top-1 accuracy (%, mean ± SD over seeds 0–2). Source S1 (Table 12). Tree-model parameter counts are storage counts: Random Forest nodes × (2 + classes), HistGradientBoosting nodes × 5. GNCA counts include the untrained auxiliary heads (Section 3.3).

Model	Params (Hermes)	Hermes $n_{\text{test}} = 3530$	BFCL (E1v2 split) $n_{\text{test}} = 692$	ToolBench $n_{\text{test}} = 422$	API-Bank $n_{\text{test}} = 483$
Majority class	1	52.86 ± 0.00	56.07 ± 0.00	36.26 ± 0.00	33.54 ± 0.00
Logistic regression	2,457	65.21 ± 0.12	74.52 ± 0.37	56.71 ± 0.49	79.23 ± 0.24
Random Forest	4,606,770	67.93 ± 0.31	74.18 ± 0.96	74.33 ± 1.12	80.13 ± 0.55
HistGradientBoosting	43,515	66.20 ± 0.32	78.18 ± 0.58	76.07 ± 1.24	84.75 ± 0.59
Random Forest, cell features	–	69.04 ± 0.35	72.64 ± 0.09	73.38 ± 0.59	81.64 ± 0.98
HistGradientBoosting, cell features	–	66.81 ± 0.70	78.18 ± 0.58	77.25 ± 0.48	85.64 ± 0.12
MLP (76 hidden)	62,713	69.91 ± 0.45	77.60 ± 0.15	74.17 ± 1.19	86.47 ± 0.73
GNCA chain	59,182	69.11 ± 0.16	76.01 ± 0.72	75.12 ± 0.63	85.09 ± 1.69
GNCA star	59,182	69.66 ± 0.18	76.30 ± 1.39	75.35 ± 1.85	87.09 ± 1.57
GNCA tree (primary)	59,182	69.10 ± 0.12	76.40 ± 0.67	76.30 ± 1.85	85.99 ± 1.38
GNCA learned	59,207	69.44 ± 0.30	76.83 ± 0.60	75.75 ± 1.07	86.75 ± 1.69

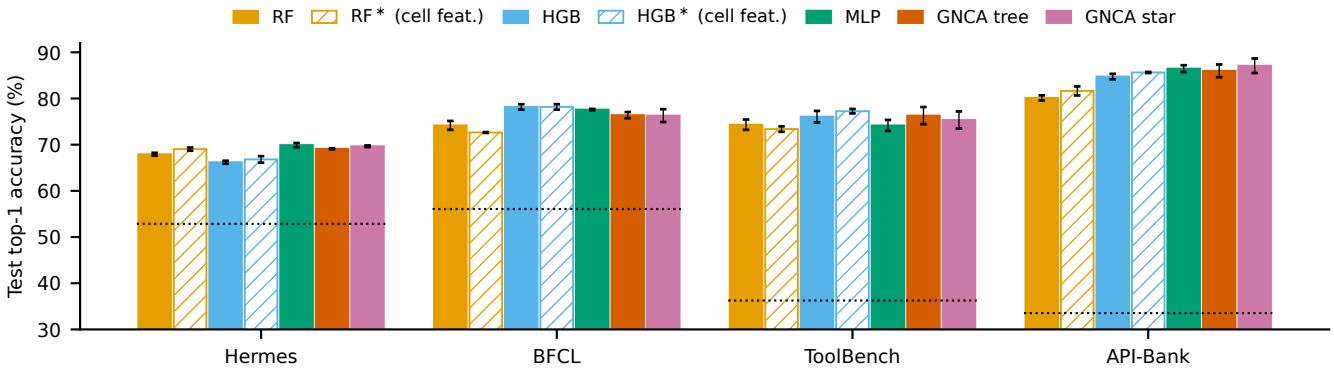


Figure 3: Test top-1 accuracy per benchmark (bars: mean over 3 seeds; whiskers: SD). Hatched bars: tree models fitted on the same cell features as the GNCA. Dotted lines: majority class.

samples a random subset of features at each split, loses accuracy on the longer, mostly zero cell vector.

Protocol sensitivity. An earlier Hermes run used a different feature pipeline for the tree models (source S3). It gave HistGradientBoosting 69.47%, Random Forest 67.88% and the star and tree GNCA 66.19% and 65.27%, the opposite ranking against gradient boosting. Which model wins on Hermes therefore depends on the features the tree models receive. On BFCL, training the GNCA for 150,000 steps instead of 800 (source S4) gives 75.29–76.40% across topologies, still below HistGradientBoosting (78.18%).

Single-seed search. Table 10 and Figure 5a show what the search changed. Tuning helped logistic regression and the tree models more than the neural models. Pooled gains are +4.04 pp for logistic regression, +0.98 pp for the Random Forest and +1.18 pp for HistGradientBoosting (CI $[+0.47, +1.90]$), and +1.72 and +1.09 pp on cell features. The MLP changes by −0.08 pp and the tree GNCA by +0.37 pp (CI $[-0.69, +1.43]$). After Holm correction 6 of the 40 per-benchmark gains are significant, all positive and none for a neural model. With one seed and 20 trials, a configuration that wins on validation can still lose a little on test (Table 10); this happens for the MLP and for three GNCA variants on some benchmarks, never significantly.

Between tuned models (Table 11, Figure 5b), 3 of the 12 primary comparisons are significant after Holm. The tuned tree GNCA is ahead of the tuned Random Forest on API-Bank (+5.11 pp, $p_{\text{Holm}} = 0.002$). It is behind tuned HistGradientBoosting and the tuned MLP on BFCL (−2.84 and −2.02 pp, $p_{\text{Holm}} = 0.027$ and 0.027). Pooled, its lead over the Random Forest shrinks from +2.55 to +1.99 pp and is no longer significant (CI $[-0.30, +4.27]$, $p = 0.089$). Against HistGradientBoosting and the MLP the pooled differences are −0.11 pp (CI $[-2.31, +2.08]$) and +0.09 pp (CI $[-1.55, +1.72]$). In the secondary family, tuned HistGradientBoosting on cell features is ahead of the GNCA on Hermes (−1.33 pp, $p_{\text{Holm}} = 0.006$) and the tuned RF* is behind it on API-

Table 8: Paired comparisons, tree GNCA minus baseline (pp). Seed-clustered bootstrap over test turns. Upper block: primary family (Holm over 12); lower block: secondary family (Holm over 8). Pooled: random-effects estimate over four benchmarks. RF: Random Forest; HGB: HistGradientBoosting; *: fitted on the GNCA's cell features.

Benchmark	Baseline	Δ (pp)	95% CI	p	p_{Holm}
Hermes	RF	+1.17	[+0.22, +2.11]	0.0155	0.1530
Hermes	HGB	+2.90	[+1.96, +3.81]	0.0001	0.0012
Hermes	MLP	-0.81	[-1.47, -0.17]	0.0153	0.1530
BFCL	RF	+2.22	[+0.05, +4.43]	0.0515	0.3605
BFCL	HGB	-1.78	[-3.71, +0.14]	0.0670	0.4020
BFCL	MLP	-1.20	[-2.84, +0.43]	0.1505	0.7524
ToolBench	RF	+1.97	[-0.87, +4.74]	0.1794	0.7524
ToolBench	HGB	+0.24	[-2.21, +2.76]	0.8812	1.0000
ToolBench	MLP	+2.13	[+0.24, +4.03]	0.0307	0.2456
API-Bank	RF	+5.87	[+3.11, +8.70]	0.0002	0.0022
API-Bank	HGB	+1.24	[-0.69, +3.31]	0.2246	0.7524
API-Bank	MLP	-0.48	[-2.07, +1.10]	0.5507	1.0000
Pooled	RF	+2.55	[+0.67, +4.43]	0.0077	–
Pooled	HGB	+0.75	[-1.50, +3.00]	0.5132	–
Pooled	MLP	-0.26	[-1.43, +0.92]	0.6686	–
Secondary: tree models on the same cell features (Holm over 8)					
Hermes	RF*	+0.07	[-0.81, +0.95]	0.8969	1.0000
Hermes	HGB*	+2.29	[+1.37, +3.21]	0.0001	0.0008
BFCL	RF*	+3.76	[+1.35, +6.12]	0.0021	0.0126
BFCL	HGB*	-1.78	[-3.71, +0.14]	0.0670	0.2680
ToolBench	RF*	+2.92	[+0.16, +5.77]	0.0445	0.2225
ToolBench	HGB*	-0.95	[-3.55, +1.66]	0.4598	1.0000
API-Bank	RF*	+4.35	[+1.73, +6.90]	0.0014	0.0098
API-Bank	HGB*	+0.35	[-1.66, +2.35]	0.7301	1.0000
Pooled	RF*	+2.60	[+0.12, +5.07]	0.0398	–
Pooled	HGB*	+0.12	[-2.02, +2.25]	0.9139	–

Bank (+4.55 pp, $p_{\text{Holm}} = 0.002$); the pooled differences are +0.98 and −0.78 pp. Choosing the tuned GNCA topology on validation would pick the star topology on Hermes (69.76% test), the star on BFCL (76.97%), the chain on ToolBench (75.75%) and the learned on API-Bank (87.23%). With multi-seed selection and five seeds (Section 6.1) the BFCL differences to HistGradientBoosting and the MLP are no longer significant, and the Hermes difference to HGB* remains.

D. Hyperparameter Search

Search spaces (log-uniform ranges are marked log):

- **Logistic regression:** inverse regularisation `C` $\in [10^{-3}, 10^{2}]$ (log); class weights none or balanced; 1,000 iterations (default: `C`=1, 300 iterations).
- **Random Forest (RF, RF*):** 50, 100, 200 or 300 trees; maximum depth 10, 15, 20, 30 or none; minimum leaf size 1, 2, 4 or 8; features per split $\sqrt{d}$, $\log_2 d$, 10% or 30%; class weights none or balanced subsample.
- **HistGradientBoosting (HGB, HGB*):** learning rate $[0.02, 0.3]$ (log); 100, 200 or 400 iterations; 15, 31, 63 or 127 leaves; minimum leaf size 5, 20 or 50; ℓ_2 penalty 0, 10^{-3}, 0.1, 1 or 10; class weights none or balanced; no early stopping.
- **Neural models (shared):** learning rate $[3 \times 10^{-4}, 6 \times 10^{-3}]$ (log); weight decay $[10^{-3}, 0.2]$ (log); label smoothing 0, 0.05 or 0.1; batch 256, 512 or 1,024; 1,500, 3,000 or 6,000 steps on Hermes and 400, 800 or 1,600 elsewhere; best-validation checkpoint as in Section 5.
- **MLP:** hidden width 38, 76, 152 or 304; dropout 0, 0.1 or 0.3.
- **GNCA (all topologies):** $K \in \{2, 4, 6, 8\}$; hidden channels $H \in \{8, 16, 32\}$; update-MLP width 48, 96 or 192.

Table 9 lists the configuration selected for each model and benchmark.

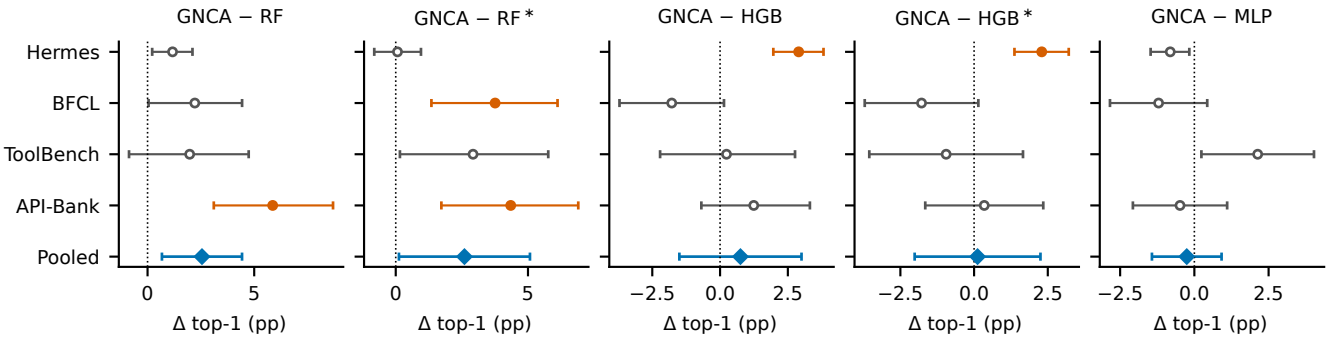


Figure 4: Paired differences from Table 8 with 95% bootstrap CIs. Filled orange: Holm-adjusted $p < 0.05$; open grey: not significant; blue diamonds: random-effects pooled estimate. Source S1.

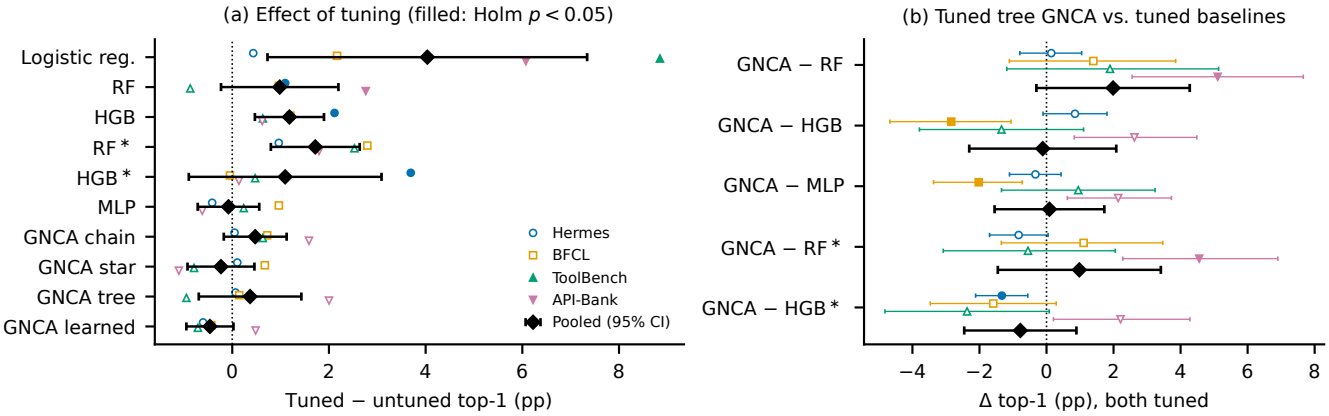


Figure 5: (a) Tuned minus default test top-1 per model: markers per benchmark (filled: Holm-adjusted $p < 0.05$ over 40), diamonds: random-effects pooled estimate with 95% CI. (b) Tuned tree GNCA minus tuned baselines with 95% bootstrap CIs (filled: Holm $p < 0.05$ within the family of Table 11); diamonds: pooled estimate. Source S6.

Table 9: Selected configurations (source S6). cw: class weights; it.: iterations; st.: training steps; hid.: hidden width; upd: update-MLP width.

Model	Trial	Selected configuration (highest seed-0 validation top-1; trial 0 = default)
Hermes		
Logistic regression	8	C=82.3, cw=none
Random Forest	7	50 trees, depth 15, leaf 1, feat 0.3, cw=none
HistGradientBoosting	14	lr 0.159, 100 it., 63 leaves, leaf 5, ℓ_2 10, cw=none
Random Forest, cell features	16	50 trees, depth 20, leaf 1, feat 0.3, cw=none
HistGradientBoosting, cell features	16	lr 0.0272, 100 it., 63 leaves, leaf 20, ℓ_2 1, cw=none
MLP	7	lr 0.0025, wd 0.01, LS 0.1, bs 256, 3,000 st., 304 hid., drop 0
GNCA chain	11	lr 0.0026, wd 0.012, LS 0.05, bs 256, 6,000 st., K=6, H=8, upd 48
GNCA star	6	lr 0.0027, wd 0.036, LS 0.05, bs 256, 6,000 st., K=8, H=16, upd 48
GNCA tree	6	lr 0.0027, wd 0.0064, LS 0.1, bs 256, 6,000 st., K=8, H=8, upd 48
GNCA learned	8	lr 0.0026, wd 0.0018, LS 0.1, bs 1,024, 6,000 st., K=6, H=8, upd 48
BFCL		
Logistic regression	1	C=62, cw=none
Random Forest	9	200 trees, depth 15, leaf 4, feat 0.3, cw=none
HistGradientBoosting	8	lr 0.206, 400 it., 63 leaves, leaf 50, ℓ_2 0.001, cw=none
Random Forest, cell features	17	100 trees, depth none, leaf 2, feat 0.3, cw=none
HistGradientBoosting, cell features	14	lr 0.145, 100 it., 15 leaves, leaf 50, ℓ_2 0.001, cw=bal.
MLP	14	lr 0.00041, wd 0.0099, LS 0.1, bs 512, 800 st., 76 hid., drop 0.3
GNCA chain	4	lr 0.0015, wd 0.0019, LS 0.1, bs 256, 800 st., K=8, H=16, upd 96
GNCA star	1	lr 0.0011, wd 0.0023, LS 0.05, bs 256, 1,600 st., K=2, H=16, upd 48
GNCA tree	19	lr 0.0037, wd 0.12, LS 0.1, bs 512, 400 st., K=6, H=16, upd 192
GNCA learned	16	lr 0.00059, wd 0.067, LS 0.05, bs 256, 1,600 st., K=6, H=32, upd 192
ToolBench		
Logistic regression	8	C=61.1, cw=none
Random Forest	5	100 trees, depth 15, leaf 2, feat 0.3, cw=none
HistGradientBoosting	1	lr 0.175, 400 it., 15 leaves, leaf 20, ℓ_2 0, cw=none
Random Forest, cell features	3	100 trees, depth 15, leaf 1, feat log2, cw=bal.
HistGradientBoosting, cell features	4	lr 0.114, 100 it., 63 leaves, leaf 50, ℓ_2 0, cw=bal.
MLP	1	lr 0.00089, wd 0.15, LS 0, bs 1,024, 1,600 st., 304 hid., drop 0.1
GNCA chain	11	lr 0.0034, wd 0.14, LS 0.1, bs 256, 1,600 st., K=8, H=16, upd 48
GNCA star	7	lr 0.0025, wd 0.16, LS 0.1, bs 256, 800 st., K=4, H=16, upd 96
GNCA tree	4	lr 0.00071, wd 0.074, LS 0.05, bs 1,024, 1,600 st., K=4, H=8, upd 192
GNCA learned	13	lr 0.00052, wd 0.0054, LS 0.05, bs 1,024, 1,600 st., K=4, H=8, upd 192
API-Bank		
Logistic regression	14	C=43, cw=none
Random Forest	8	100 trees, depth 20, leaf 2, feat 0.1, cw=bal.
HistGradientBoosting	8	lr 0.22, 400 it., 63 leaves, leaf 50, ℓ_2 0.1, cw=bal.
Random Forest, cell features	2	300 trees, depth none, leaf 2, feat 0.3, cw=none
HistGradientBoosting, cell features	5	lr 0.079, 200 it., 63 leaves, leaf 20, ℓ_2 0, cw=bal.
MLP	3	lr 0.00094, wd 0.011, LS 0.1, bs 1,024, 800 st., 38 hid., drop 0
GNCA chain	6	lr 0.0028, wd 0.01, LS 0.1, bs 256, 1,600 st., K=6, H=32, upd 192
GNCA star	11	lr 0.0027, wd 0.01, LS 0.05, bs 1,024, 1,600 st., K=8, H=16, upd 192
GNCA tree	18	lr 0.0042, wd 0.0085, LS 0.1, bs 512, 800 st., K=2, H=16, upd 192
GNCA learned	8	lr 0.004, wd 0.044, LS 0.05, bs 256, 1,600 st., K=6, H=8, upd 96

Table 10: Tuned models: test top-1 accuracy (%, mean ± SD over seeds 0–2) after the per-benchmark search, and Δ_{tune}, tuned minus default (pp, seed-clustered bootstrap; †: Holm-adjusted $p < 0.05$ over 40). Pooled: random-effects estimate over the four benchmarks. Source S6 (Table 12).

	Hermes		BFCL		ToolBench		API-Bank		Pooled
Model	Tuned	Δ_{tune}	Tuned	Δ_{tune}	Tuned	Δ_{tune}	Tuned	Δ_{tune}	Δ_{tune}
Logistic regression	65.65 ± 0.07	+0.43	76.69 ± 0.33	+2.17	65.56 ± 1.58	+8.85†	85.30 ± 0.55	+6.07†	+4.04
Random Forest	69.03 ± 0.22	+1.10†	75.15 ± 0.25	+0.96	73.46 ± 0.63	-0.87	82.89 ± 0.52	+2.76†	+0.98
HistGradientBoosting	68.32 ± 0.23	+2.12†	79.39 ± 0.85	+1.20	76.70 ± 0.59	+0.63	85.37 ± 0.78	+0.62	+1.18
Random Forest, cell features	70.00 ± 0.28	+0.96	75.43 ± 0.29	+2.79	75.91 ± 0.55	+2.53	83.44 ± 0.36	+1.79	+1.72
HistGradientBoosting, cell features	70.50 ± 0.12	+3.69†	78.13 ± 0.47	-0.05	77.73 ± 0.72	+0.47	85.78 ± 0.66	+0.14	+1.09
MLP	69.50 ± 0.28	-0.42	78.57 ± 0.17	+0.96	74.41 ± 0.63	+0.24	85.85 ± 0.67	-0.62	-0.08
GNCA chain	69.16 ± 0.43	+0.05	76.73 ± 0.29	+0.72	75.75 ± 0.14	+0.63	86.68 ± 0.63	+1.59	+0.47
GNCA star	69.76 ± 0.07	+0.10	76.97 ± 0.22	+0.67	74.56 ± 1.10	-0.79	85.99 ± 0.63	-1.10	-0.23
GNCA tree (primary)	69.17 ± 0.13	+0.07	76.54 ± 0.93	+0.14	75.36 ± 0.95	-0.95	87.99 ± 0.42	+2.00	+0.37
GNCA learned	68.84 ± 0.34	-0.60	76.40 ± 0.33	-0.43	75.04 ± 1.58	-0.71	87.23 ± 0.24	+0.48	-0.46

Table 11: Paired comparisons after tuning, tuned tree GNCA minus tuned baseline (pp). Same tests and families as Table 8. Source S6.

Benchmark	Baseline	Δ (pp)	95% CI	p	p_{Holm}
Primary family (Holm over 12)					
Hermes	RF	+0.14	[-0.79, +1.05]	0.7705	1.0000
Hermes	HGB	+0.85	[-0.10, +1.80]	0.0812	0.5683
Hermes	MLP	-0.33	[-1.10, +0.43]	0.4002	1.0000
BFCL	RF	+1.40	[-1.11, +3.85]	0.2667	1.0000
BFCL	HGB	-2.84	[-4.67, -1.06]	0.0027	0.0275
BFCL	MLP	-2.02	[-3.37, -0.72]	0.0025	0.0275
ToolBench	RF	+1.90	[-1.18, +5.13]	0.2390	1.0000
ToolBench	HGB	-1.34	[-3.79, +1.11]	0.2895	1.0000
ToolBench	MLP	+0.95	[-1.34, +3.24]	0.4495	1.0000
API-Bank	RF	+5.11	[+2.55, +7.66]	0.0002	0.0024
API-Bank	HGB	+2.62	[+0.83, +4.49]	0.0057	0.0513
API-Bank	MLP	+2.14	[+0.62, +3.73]	0.0073	0.0584
Pooled	RF	+1.99	[-0.30, +4.27]	0.0889	–
Pooled	HGB	-0.11	[-2.31, +2.08]	0.9202	–
Pooled	MLP	+0.09	[-1.55, +1.72]	0.9164	–
Secondary: tuned trees on cell features (Holm over 8)					
Hermes	RF*	-0.83	[-1.70, +0.05]	0.0594	0.2905
Hermes	HGB*	-1.33	[-2.12, -0.56]	0.0008	0.0056
BFCL	RF*	+1.11	[-1.35, +3.47]	0.3601	0.7201
BFCL	HGB*	-1.59	[-3.47, +0.29]	0.0916	0.2905
ToolBench	RF*	-0.55	[-3.08, +2.05]	0.6906	0.7201
ToolBench	HGB*	-2.37	[-4.82, +0.08]	0.0581	0.2905
API-Bank	RF*	+4.55	[+2.28, +6.90]	0.0003	0.0024
API-Bank	HGB*	+2.21	[+0.21, +4.28]	0.0327	0.1962
Pooled	RF*	+0.98	[-1.45, +3.41]	0.4300	–
Pooled	HGB*	-0.78	[-2.45, +0.89]	0.3601	–

E. Result Sources and Changes from Version 7.6

Table 12 names the result sources used in this paper. Version 7.6 contained claims that no result file supports, or that the code contradicts. Table 13 lists what was removed or replaced. The repository file `paper/LEDGER_V76C_1110.md` gives the exact file and key for every entry.

Table 12: Result sources. Directories are relative to `stage0/stage0_results/`.

Id	Directory	Content
S1	`review_1110/`	Four-benchmark accuracies, per-turn correctness, paired statistics, input-matched trees, CPU cost, Jacobian audit, Lean axiom log
S2	top level	Hermes factorial run (reviewer suite): depth, topology, loss, carry, halting, contractive variant, escalation, learning curve
S3	top level	Earlier full-history Hermes run with a different tree-model feature pipeline
S4	`e1v2/run/`	BFCL long-training run (150,000 steps)
S5	`baselines_20261026/`	Local LLM routing baselines (Qwen3.5-9B, Gemma-4-12B)
S6	`hpo_1110/`	Hyperparameter search: per-trial logs, selected configurations, tuned refits (3 seeds), per-turn correctness, paired statistics, software environment
S7	`hermes_ood_20261026/`	Earlier out-of-distribution pilot on private Hermes sessions (summary statistics only; superseded by S8)
S8	`rigour_1110/`	Main comparison: multi-seed selection, five-seed finals of 13 models, per-turn correctness and probabilities, OOD split, ablations, inference and featurisation latency, refit check

Table 13: Ledger of Version 7.6 claims and their replacements (sources: Table 12).

Version 7.6 claim	Action	Replacement and source
Title "From Grokking to . . . "	Removed	Gradual learning curve, 68.15 → 70.79% best-so-far validation (S2)
Title ". . . for Sub-Millisecond Agent Tool Selection" (aiXiv v1.5)	Kept, scoped	"Sub-millisecond" refers to inference: 0.559 ms for the tuned tree GNCA on one CPU thread (S8). Featurisation (7.65 ms, unoptimised Python) is reported separately as preprocessing
Headline K=8 GNCA (68.92%) and "statistically non-inferior" to HistGradientBoosting (TOST)	Removed	Pre-specified K=4 tree; difference tests only (Table 8, S1)
Per-turn bootstrap over seeds × turns, Cohen's d/h, power 0.94, linear mixed-effects +3.16 pp	Replaced	Seed-clustered paired bootstrap, Holm, DerSimonian–Laird (S1)
"Pre-registered" confirmatory claims	Removed	No pre-registration exists; primary family stated in Section 5
50-trial Hyperband tuning of all models	Replaced	No Hyperband code or result exists; equal-budget random search, 20 trials per model and benchmark (Section 5, Appendix D, S6)
Escalation weight β=0.5, "calibrated" score with range $[0, 1.50]$, 10-bin reliability audit	Replaced	β=1 as in the evaluation script; AUROC and two operating points (S2); no calibration result exists
Damped update with α_d, γ and Bernoulli(p=0.8) cell masks	Replaced	Update rule of the model code (Eq. (4))
Causal lower-triangular adjacency	Replaced	Undirected row-normalised `build_adjacency` (Section 3.2)
$\alpha_{\text{self}} \times K$ grid; "ablation" and "Pareto" figures	Removed (values hard-coded in plot scripts)	Factorial table (Table 6, S2)
Contractive variant "satisfies the dyadic premise on 100% of turns", $\rho \leq 0.47$, ratio max 0.437; "$L_{\text{deploy}} \leq 0.8$"	Removed	The audit keys are not written by any repository code; accuracy 61.99% kept (S2); Jacobian audit (S1)
Bridge constants (Jacobian 0.887/0.890, 62,645 turns, radius 5.14, 1.64)	Removed	No result file
"Ten theorems with zero axioms"	Replaced	28 theorems, 13 axiom-free, rest standard axioms (S1)
Tree concordance 76.63%, $\kappa = 0.5628$, as evidence for the tree-simulation theorem	Removed	The value is in a top-level audit file, but no repository script produces it, and agreement with a separately fitted tree does not test the theorem
GCN, LGCA, E(n)-GNCA, Translucent, TextNCA, ARTS, Contriever, Hammer baselines	Removed	No results in the repository; prior work cited in Section 2
Transformer "1.2M parameters, 65.87%, 3.8 ms"	Removed	An earlier-protocol Transformer (276,749 parameters, 65.87 ± 2.00%) is in S5, but it was not run on the four-benchmark protocol; the 1.2M and 3.8 ms figures have no file
Qwen2.5-1.5B constrained decoding 55.20%; frontier-API latencies	Removed	Local LLM numbers kept: Qwen 35.01%/378 ms, Gemma 38.02%/848 ms (S5)
Linear classifier 60.71%; kFLOPs, RAM 142.6 MB	Replaced	Logistic regression row (S1); serialised sizes (S1)
Synthetic 10k-trajectory and probe-leakage numbers	Removed	No result file; leakage handled analytically (Proposition 2)
±25%/±50% hyperparameter sensitivity analyses	Removed	No code or result file; replaced by the search of Section 5 (S6)
Agent-loop pilot statistics (pass rates, token savings, speedups)	Removed	Samples too small, no matched comparison; turn-level evaluation only (Section 7)
Intel Xeon 8375C latency (21.2 ms, 60.6×), multi-hardware audit, 18.2M Random-Forest nodes	Replaced	AMD Ryzen 9 7950X 16-Core Processor latencies, 307,118 nodes (S1)
Hermes HistGradientBoosting 69.47% as the main comparison	Kept as sensitivity	S3; main table from S1
Parameter breakdown with inconsistent totals (8,846 invariant part)	Replaced	Eq. (5), asserted against S1 by the table generator
Hugging Face model link; references without a verifiable record	Removed	Repository link; bibliography checked against arXiv and Crossref